COMPOSITE MECHANICS / STUDY GUIDE

Theory and line-by-line code explanation

Composite Modulus Python Project

A detailed walkthrough of 24 files, accounting for 1,120 nonblank source lines.

Understand the physical quantities, follow the six analytical model families, and connect each equation to its implementation.

Includes numbered source listings, line commentary, a complete worked numerical trace, tests and output interpretation.

Scope: isotropic spherical-inclusion calculations with illustrative inputs. Woven architecture and experimental validation are not included.

Use the contents page or PDF bookmarks to jump to a file. Code lines that wrap retain their original source number on the first display line.

Contents

1. What the project is actually doing 4

2. How the folders cooperate 5

3. Python notation you will repeatedly see 6

4. Kelvin tensors: the concept behind the matrix code 6

5. File-by-file annotated source 7

requirements.txt 7

README.md 9

utilities/input_checks.py 15

utilities/elastic_properties.py 18

utilities/tensor_operations.py 20

models/_common.py 21

models/rule_of_mixtures.py 23

models/eshelby.py 25

models/mori_tanaka.py 29

models/self_consistent.py 31

models/hashin_shtrikman.py 35

models/halpin_tsai.py 37

models/__init__.py 38

data/materials.json 40

utilities/materials.py 43

analysis/compare_models.py 48

analysis/plotting.py 53

main.py 58

tests/test_limits.py 64

tests/test_benchmarks.py 69

tests/test_project.py 74

utilities/__init__.py 78

analysis/__init__.py 79

tests/__init__.py 79

6. A complete numerical trace: following one calculation 80

7. Understanding outputs/figures and outputs/tables 82

8. The report folder beyond the originally pasted tree 84

9. What you should be able to explain in your presentation 85

10. Sources for further study 86

11. Source-version and coverage record 87

1. What the project is actually doing

We replace a complicated two-phase material with an equivalent homogeneous elastic material. This is **homogenization**. We predict the elastic response of an idealized composite; we do not manufacture or test a specimen in this program. A model supplies a relationship between assumed microscopic structure and macroscopic stiffness.

The 3D inclusion calculations in this version assume isotropic solid phases, spherical inclusions, perfect bonding, small strain and linear elasticity. The symbol `f` means the reinforcement/inclusion phase. It does not mean the program is geometrically modeling an actual long fiber. The matrix is phase `m`. Real cylindrical fibers and woven fabric need additional geometry and often anisotropic properties. The shipped numbers are explicitly illustrative.

The physical quantities

Symbol / code	Meaning	How to understand it
sigma	Stress, force divided by area	How strongly the material is loaded
epsilon	Strain, extension divided by original length	Relative deformation
E	Young's modulus	Uniaxial elastic stiffness: sigma = E epsilon
nu	Poisson ratio	Negative transverse strain divided by axial strain in a uniaxial test
K	Bulk modulus	Resistance to change of volume under hydrostatic loading
G	Shear modulus	Resistance to change of shape under shear
Vf, Vm	Phase volume fractions	Fractions of total volume, not mass fractions
Cf, Cm	Phase stiffness tensors	3D rules mapping strain to stress
S	Eshelby tensor in this project	Geometry/host constraint on transformation eigenstrain
A	Strain-concentration tensor	Converts a reference strain into inclusion strain
xi	Halpin-Tsai parameter	Requires a geometry/loading/calibration justification

`E` is not strength. A material can be stiff but fail at a low strain. This project does not calculate failure stress, toughness or fatigue life.

For a void-free two-phase material, `Vf + Vm = 1`. Enter 30% as `0.30`, not `30`. If your experiment gives mass fractions, convert using densities first:

```
Vf = (wf/rho_f) / [(wf/rho_f) + (wm/rho_m)]
```

The code has no density inputs and does not perform this mass-to-volume conversion automatically. All phase moduli must already use the same unit.

Why we convert E and nu to K and G

An isotropic solid has two independent elastic constants. Knowing E alone does not specify its full 3D behavior. The conversion is:

```
K = E/[3(1-2nu)]      G = E/[2(1+nu)]
E = 9KG/(3K+G)       nu = (3K-2G)/[2(3K+G)]
```

The sphere calculations separate volume change and shape change, so K and G are particularly convenient. The program then converts their effective values back to E, which is your main requested output. These conversions assume isotropy and cannot replace a full anisotropic fiber stiffness description.

The six families do different jobs

Family	Its job in this project
Rule of Mixtures	Simple E baselines, plus separate proper 3D isotropic bounds
Eshelby / dilute	Inclusion tensor and a low-concentration estimate
Mori-Tanaka	Matrix-host mean-field prediction using the Eshelby calculation
Classical self-consistent	Effective-medium prediction from implicit equations
Hashin-Shtrikman	Applicable isotropic upper/lower bounds, not one prediction
Halpin-Tsai	Separate semi-empirical E demonstration with explicit xi

The host is the surrounding material assumed by an inclusion model. Dilute uses an undisturbed matrix; Mori-Tanaka uses average matrix strain; classical self-consistent uses an unknown effective composite medium. See the original project references for the mathematical sources. No comparison plot alone shows which model best matches a real specimen.

2. How the folders cooperate

```
main.py
  -> utilities/materials.py reads data/materials.json
  -> utilities/input_checks.py validates the inputs
  -> analysis/compare_models.py selects and evaluates the models
       -> models/*.py use elastic_properties.py and tensor_operations.py
  -> analysis/plotting.py draws already-computed results
  -> outputs/tables stores CSV rows
  -> outputs/figures stores graphs
  -> report/build_report.py runs verification and creates the report (if requested)

tests/*.py -> independently check formulas, limits and program behavior
```

Your best learning order is validation -> elastic conversions -> mixture formulas -> tensor basics -> Eshelby -> Mori-Tanaka -> self-consistent -> HS -> Halpin-Tsai -> orchestration/plots -> tests. `main.py` is the starting point when running the program, but it is not where most of the mechanics lives.

3. Python notation you will repeatedly see

Syntax	Meaning in this project
`def name(...):`	Defines a function; its body runs when called
`return`	Sends the computed value back to the caller
`a, b = ...`	Unpacks two returned values into two variables
`_`	A valid variable name used when the result is intentionally ignored
`Vm=None`	Optional argument; absence means compute the matrix complement
`*, nu_f, nu_m`	Arguments after the star must be named when calling the function
`**inp` in a call	Unpacks a dictionary into named function arguments
`{**inp, 'Vf': .02}`	Copies a dictionary and replaces its Vf entry
`*values` in a call	Unpacks a tuple/list into positional arguments
`*` with NumPy arrays	Elementwise multiplication; also scalar scaling
`@` with NumPy arrays	Matrix multiplication, used for tensor composition
`np.eye(6)`	6-by-6 identity matrix
`np.linalg.solve(M, B)`	Finds X satisfying M X = B; avoids forming an explicit inverse
`.T`	Matrix transpose
`np.sum(J*C)`	Sum of elementwise products; a Frobenius inner product
`f'{value:.6f}'`	Formats a number with six digits after the decimal point
`if __name__ == '__main__':`	Runs the entry-point block only when the file is executed directly
`raise ValueError(...)`	Rejects an invalid value with a readable explanation
`try / except`	Attempts an operation and handles specified failures
`from .module import ...`	Imports from another file in the same package
`Path` and `/`	Build filesystem paths; `/` here is path joining, not division

Imports make definitions available; importing a function does not itself run a material calculation. `__init__.py` files expose package interfaces so that `from models import mori_tanaka` can work from the project folder.

4. Kelvin tensors: the concept behind the matrix code

A symmetric 3D strain tensor has six independent components. This project stores it as `[e11,e22,e33,sqrt(2)e23,sqrt(2)e13,sqrt(2)e12]` and stores stress with the same scaling. This orthonormal convention preserves tensor inner products. It differs from engineering-Voigt notation, where engineering shear strain is often doubled. You must not mix matrices from the two conventions.

For example, pure shear obeys `sigma23 = 2G e23 = G gamma23`, where `gamma23 = 2e23`. Therefore the Kelvin shear stiffness entries are `2G`, not the engineering-Voigt shear entries `G`.

Split any strain into a hydrostatic (volume-changing) part and a deviatoric (shape-changing) part. `J` and `D` are projection matrices for these parts:

```
h = [1,1,1,0,0,0]/sqrt(3)
J = h h^T       D = I-J
C = 3K J + 2G D
```

The hydrostatic subspace has dimension 1; the deviatoric subspace has dimension

5. Thus `J:J=1`, `D:D=5`, and `J:D=0`. It follows that `K=(J:C)/3` and `G=(D:C)/10`. That explains both denominators in `properties_from_stiffness`. The extractor is intended for an isotropic C produced by these models. It does not check that an arbitrary supplied anisotropic matrix is actually isotropic.

5. File-by-file annotated source

The code blocks below are numbered transcripts, not replacement files to paste into Python. Every chapter first explains purpose and theory, then gives source and the line commentary. File hashes at the end identify the exact version. Long source lines are visually wrapped in this PDF; the original source line numbering remains unchanged.

requirements.txt

Source file: `requirements.txt`

Purpose

This file lists external Python dependencies for pip. It is not a Python program. Standard-library modules such as math, json, csv, pathlib, argparse and unittest are already included with Python and therefore are not listed. Version ranges allow compatible installations; they are not exact locks. requirements-tested.txt records the exact top-level versions used for the shipped outputs. Matplotlib and ReportLab are loaded only when their workflows are requested, whereas the basic model calculations require NumPy.

Numbered source

```
001 | # Core scientific computations and static plots
002 | numpy>=1.24,<3
003 | matplotlib>=3.7,<4
004 | # Required only when creating the PDF report
005 | reportlab>=4.0,<5
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	A comment grouping the core numeric/plotting dependencies; pip ignores text after #.
L2	Requests NumPy version at least 1.24 and below 3. It supplies arrays, tensor products, linear solves and numerical comparisons.
L3	Requests Matplotlib version at least 3.7 and below 4. It draws the computed curves and exports PNG/PDF charts; it does not predict E.
L4	A comment explaining that the next library serves optional PDF report generation.
L5	Requests ReportLab version at least 4 and below 5. It lays out report paragraphs, tables and figures; it is not used by the mechanics equations.

README.md

Source file: `README.md`

Purpose and how to read its lines

README is the operating manual. Markdown headings, lists, tables and fenced code blocks format text; they are not executed as project code. Commands in shell blocks are instructions you can choose to run in a terminal. The comments in the folder diagram describe the files explained in the other chapters of this guide. Those diagram lines do not create files or directories.

Numbered source

````
001 | # Composite Modulus: analytical major-project framework
002 |
003 | Predict effective elastic Young's modulus using six model families. This is
004 | a complete runnable implementation of the agreed **isotropic spherical-inclusion
005 | scope**, with illustrative material studies, reproducible results and a report.
006 | It does not claim to predict woven architecture or validate a real biocomposite.
007 |
008 | ## Quick start in VS Code
009 |
010 | 1. Extract the complete project ZIP and open **the folder containing main.py**
011 |    using File > Open Folder. Keep all project folders together.
012 | 2. Install Python 3.10+ and Microsoft's Python extension.
013 | 3. Use Python: Create Environment > Venv and select it with Python: Select
014 |    Interpreter. Open a new VS Code terminal in the project folder.
015 | 4. Install dependencies and run:
016 |
017 | ```sh
018 | python -m pip install -r requirements.txt
019 | python main.py
020 | ```
021 |
022 | On macOS/Linux, if your command is `python3`, use that spelling consistently.
023 | If desired, create and activate the environment manually:
024 |
025 | ```sh
026 | # macOS / Linux
027 | python3 -m venv .venv
028 | source .venv/bin/activate
029 | python -m pip install -r requirements.txt
030 | ```
031 |
032 | ```powershell
033 | # Windows PowerShell
034 | py -3 -m venv .venv
035 | .\.venv\Scripts\Activate.ps1
036 | python -m pip install -r requirements.txt
037 | ```
038 |
039 | If PowerShell blocks activation, select `.venv` through VS Code instead or use
040 | `.\.venv\Scripts\python.exe` directly in place of `python`; changing the
041 | machine's script policy is unnecessary for this project.
042 |
043 | ## Generate all results and the report
044 |
045 | ```sh
046 | python main.py --all --report
047 | ```
048 |
049 | This regenerates three material sweeps, CSV result tables, six charts (each
050 | available as PNG and vector PDF), a Markdown/PDF report, a manifest and actual
051 | test output. It runs the tests before producing the report. The shipped outputs
052 | already contain a completed default run, so you can inspect them immediately.
053 |
054 | Other useful commands:
055 |
056 | ```sh
````

```
python main.py --list-cases
python main.py --case stiff_inclusions --vf 0.05 --vm 0.95
python main.py --compare --case moderate_contrast
python main.py --all --vf-max 1 --points 101
python main.py --case stiff_inclusions --vf 0.05 --xi 3
python -m unittest discover -s tests -v
```

`--vf` and `--vm` apply to single runs. For sweeps use `--vf-max`. The default sweep is 0 to 0.6. Phase endpoints are mathematical checks, not evidence of physical accuracy at high inclusion content. Key fractions are added to the requested plot grid automatically. `--output PATH` changes the result folder. Relative paths supplied at the command line are relative to your terminal's working directory; built-in defaults resolve relative to the project itself.

VS Code's Run and Debug menu includes **Run one material case** and **Generate all results and report** configurations. Run `main.py`, not a model file directly. The model files use package imports.

Folder structure

```text
composite_modulus/
├── main.py
├── requirements.txt
├── requirements-tested.txt       # Versions used to generate shipped results
├── README.md
├── .gitignore
├── .vscode/
│   ├── launch.json
│   └── settings.json
├── data/
│   └── materials.json
├── models/
│   ├── __init__.py
│   ├── _common.py                # Compatibility imports from utilities
│   ├── rule_of_mixtures.py
│   ├── eshelby.py
│   ├── mori_tanaka.py
│   ├── self_consistent.py
│   ├── hashin_shtrikman.py
│   ├── halpin_tsai.py
│   └── README.md
├── utilities/
│   ├── __init__.py
│   ├── input_checks.py
│   ├── elastic_properties.py
│   ├── tensor_operations.py
│   └── materials.py
├── analysis/
│   ├── __init__.py
│   ├── compare_models.py
│   └── plotting.py
├── tests/
│   ├── __init__.py
│   ├── test_limits.py
│   ├── test_benchmarks.py
│   └── test_project.py
├── report/
│   ├── __init__.py
│   ├── build_report.py
│   ├── theory.md
│   ├── background_and_viva.md
│   ├── references.md
│   └── discussion.md              # Generated from actual results
└── outputs/
    ├── figures/                  # PNG and vector PDF graphs
    ├── tables/                   # CSV: all cases and key fractions
    ├── report/                   # Generated Markdown and PDF report
    ├── run_manifest.json
    ├── verification.json
    └── verification.txt
```

```
|
| The compatibility `_common.py` preserves the previously delivered model
| interfaces; shared calculations now live in `utilities`. Keep these folders
| together. This replaces the earlier standalone-models archive as the complete
| project distribution.
|
| ## Enter your actual material data
|
| Edit `data/materials.json`, or copy it and pass `--data your_file.json`.
| Every case must supply these fields:
|
| | Field | Meaning |
| |---|---|
| | id | Unique filename-safe identifier |
| | name | Descriptive label |
| | Ef, Em | Positive phase Young's moduli in the file's common unit |
| | nu_f, nu_m | Phase Poisson ratios, strictly between -1 and 0.5 |
| | xi | Explicit nonnegative Halpin-Tsai demonstration/calibration parameter |
| | geometry | Must be `spherical` for this implementation |
| | source_status | Honest description, e.g. illustrative or literature |
| | source_note | Actual data source and any modeling assumptions |
|
| The file has schema_version=1, unit (Pa, MPa or GPa), and a materials list.
| The three shipped cases are hypothetical stiffness-contrast studies. They are
| **not measured glass/epoxy, natural-fiber or other named material datasets**.
| Neither fiber names nor property sources are invented.
|
| ```sh
| python main.py --data data/materials.json --case stiff_inclusions --compare
| ```
|
| For a real natural fiber, one scalar E and nu may be an insufficient material
| description. Decide whether an isotropic idealization is appropriate before
| entering such data; the program does not turn fibers into spheres physically.
|
| ## Reading the results
|
| - The main plot compares 3-D isotropic Voigt/Reuss bounds, HS bounds,
|   Mori-Tanaka and self-consistent predictions under a consistent spherical case.
| - A separate plot compares low-fraction dilute results and separately shows
|   the simple E mixtures and an uncalibrated Halpin-Tsai parameter demonstration.
| - Dilute rows above Vf=0.1 are blank and marked `not_evaluated`; this cutoff
|   is educational, not a universal validity threshold.
| - Cross-ordered phase K/G produces explicit `not_applicable` HS rows. It does
|   not stop valid calculations of the other models.
| - CSV files retain E, nu, K, G, status, assumptions, units and solver diagnostics.
|   Blank cells mean unavailable, not zero.
| - The run manifest stores the actual inputs and configured comparison scope.
| - The scalar arithmetic E mixture is not the general 3-D Voigt upper E bound
|   when constituent Poisson ratios differ.
| - MT and the appropriate HS boundary can coincide. That is expected for these
|   ordered isotropic spherical cases.
|
| ## Verification and report use
|
| The included suite checks endpoints, identical phases, conversions, tensor
| conventions, independent scalar formulas, an exact rational fixture, 120
| seeded material cases, self-consistent residuals, phase exchange, unit scaling,
| applicable bounds, input validation, CSV export and command-line behavior.
| See the shipped verification log for the actual result, not an assumed pass.
|
| The report contains the objective, equations, assumptions, numerical results,
| discussion, composite classifications, woven examples and references. Add your
| own institution's cover details only if required. The viva guide explains
| the choices you should understand before presenting the work.
|
| **Remaining scientific scope:** real biocomposite data and experimental
| validation are not supplied. Cylindrical/anisotropic fibers and woven
| architecture need additional models; FEM was deliberately outside this stage.
| Discuss that scope with your professor before presenting this as a prediction
| of a specific woven or natural-fiber specimen.
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	The Markdown document title identifying the project.
L3-6	Defines the project's completed computational scope and distinguishes illustrative spherical studies from real woven/biocomposite validation.
L8	Starts the VS Code quick-start section.
L10-11	Instructs you to open the project root, not only the models folder, so imports can find sibling packages.
L12	Identifies Python and Microsoft's Python extension as separate prerequisites.
L13-14	Explains creating and selecting a project-specific virtual environment, then opening its terminal.
L15	Introduces the install/run commands.
L17	Starts a shell-command code block; the fence is Markdown formatting, not part of a terminal command.
L18	Runs pip through the selected interpreter to install the dependency list, avoiding confusion between different Python installations.
L19	Runs the default single-case workflow, not the full sweep/report workflow.
L20	Ends the shell block.
L22-23	Explains Python command spelling and offers manual environment setup as an alternative to VS Code's environment command.
L25	Starts the macOS/Linux terminal example.
L26	Shell comment labeling the operating-system example.
L27	Creates an isolated environment directory named .venv.
L28	Activates that environment in the current shell.
L29	Installs dependencies into the active environment.
L30	Ends the shell example.
L32	Starts the PowerShell example.
L33	Comment identifying the Windows setup variant.
L34	Creates a virtual environment using the Windows Python launcher.
L35	Activates the environment using its PowerShell script.
L36	Installs the same project dependencies through the activated interpreter.
L37	Ends the PowerShell example.
L39-41	Offers VS Code interpreter selection or direct environment-Python execution if script activation is blocked, without changing machine-wide script policy.
L43	Starts the full-results workflow section.
L45	Starts the terminal code block for the complete run.

Source line(s)	Explanation and connection to the project
L46	Selects all material cases, performs sweeps and requests the report, which also runs verification.
L47	Ends the command block.
L49-52	Describes the generated results and clarifies that the delivered project already includes outputs from an actual run.
L54	Introduces additional usage examples.
L56	Starts another terminal example block.
L57	Lists case identifiers, phase inputs and provenance without calculating predictions.
L58	Runs one material at Vf=.05 while explicitly checking the supplied complementary Vm=.95.
L59	Sweeps the selected moderate-contrast case.
L60	Sweeps all cases through the mathematical Vf=1 endpoint on a requested 101-point grid; dilute remains unavailable above its display interval.
L61	Runs a single case with xi changed to 3; this changes the HT demonstration parameter, not the other analytical methods.
L62	Runs test discovery and reports every test method with verbose output.
L63	Ends the commands block.
L65-70	Explains single versus sweep fraction options, key-fraction insertion, output-folder selection and the distinction between user-relative and built-in paths.
L72-74	Explains the prepared VS Code debug configurations and why main.py is the entry point rather than a model file.
L76	Introduces the file-layout reference.
L78	Starts a plain-text diagram, not executable Python.
L79-128	Lists the actual project files and generated-output locations. Each path's role is explained in the corresponding guide chapter; the drawing itself creates nothing.
L129	Ends the folder-layout code block.
L131-134	Explains the compatibility import file and why models and utilities must now remain together.
L136	Starts instructions for replacing hypothetical inputs with justified material data.
L138-139	Explains editing the shipped database or selecting a copied JSON file with --data.
L141-142	Defines the header and separator of the material-field documentation table.
L143	Explains the unique identifier used for selection and filenames.
L144	Explains the descriptive display name.
L145	Requires positive phase E values in the same declared unit.
L146	Requires explicit stable isotropic Poisson ratios.
L147	Requires an explicit, justified HT parameter rather than an implicit universal default.
L148	States the geometry this implementation actually supports.

Source line(s)	Explanation and connection to the project
L149	Explains the source-status label.
L150	Requires the actual source and modeling-assumption explanation.
L152-155	Describes the root JSON schema and explicitly identifies the shipped cases as hypothetical, not named measured materials.
L157	Starts a material-file selection example.
L158	Shows choosing both the data path and case and requesting a comparison sweep.
L159	Ends the command block.
L161-163	Warns that one isotropic E,nu pair may not describe an actual anisotropic natural fiber and that input naming does not change the modeled geometry.
L165	Starts the output-interpretation section.
L167-168	Identifies the compatible 3D estimates/bounds shown on the main chart.
L169-170	Explains why dilute and the scalar/HT demonstrations appear separately.
L171-172	Explains missing dilute rows and the educational nature of the cutoff.
L173-174	Explains unavailable HS bounds when phase ordering is unsupported, while other valid calculations can continue.
L175-176	Describes CSV metadata and clarifies that blank means unavailable rather than zero.
L177	Explains the manifest's role in documenting inputs and assumptions.
L178-179	Distinguishes the simple arithmetic E formula from the 3D isotropic Voigt bound.
L180-181	Explains expected MT/HS overlap rather than promising distinct curves for every family.
L183	Starts the verification/report-use section.
L185-189	Lists the implemented checks and directs the reader to actual test output rather than an unverified claim of success.
L191-194	Describes report content and the accompanying viva guide without inventing institution-specific details.
L196-200	States remaining scientific work: real material inputs, experimental validation and geometry/anisotropy extensions. FEM remains outside the agreed analytical stage.

utilities/input_checks.py

Source file: `utilities/input_checks.py`

Purpose and physical reasoning

Every model relies on meaningful scalar inputs. A negative modulus or a volume fraction above one does not represent a supported stable two-phase solid. This module checks numerical validity before mechanics is attempted. It checks scalar values, not physical measurement quality or unit conversions. A positive Ef written in MPa and Em written in GPa would still pass unless you first put them in a common unit. `name` labels the offending input in errors. For isotropic stable solids E>0 and -1<nu<0.5; exactly incompressible solids and void phases are outside this implementation.

Numbered source

```
001 | """Validation shared by all models; no silent property assumptions."""
002 | import math
003 | import numpy as np
004 |
005 |
006 | def finite_scalar(value, name):
007 |     if isinstance(value, (bool, np.bool_)) or np.ndim(value) != 0:
008 |         raise ValueError(f"{name} must be a finite real scalar.")
009 |     try:
010 |         value = float(value)
011 |     except (TypeError, ValueError, OverflowError) as exc:
012 |         raise ValueError(f"{name} must be a finite real scalar.") from exc
013 |     if not math.isfinite(value):
014 |         raise ValueError(f"{name} must be finite.")
015 |     return value
016 |
017 |
018 | def positive(value, name):
019 |     value = finite_scalar(value, name)
020 |     if value <= 0:
021 |         raise ValueError(f"{name} must be greater than zero.")
022 |     return value
023 |
024 |
025 | def poisson(value, name):
026 |     value = finite_scalar(value, name)
027 |     if not -1 < value < 0.5:
028 |         raise ValueError(f"{name} must satisfy -1 < nu < 0.5.")
029 |     return value
030 |
031 |
032 | def fractions(Vf, Vm=None):
033 |     Vf = finite_scalar(Vf, "Vf")
034 |     if not 0 <= Vf <= 1:
035 |         raise ValueError("Vf must lie between 0 and 1 (not a percentage).")
036 |     if Vm is not None:
037 |         Vm = finite_scalar(Vm, "Vm")
038 |         if not 0 <= Vm <= 1 or not math.isclose(Vf + Vm, 1, rel_tol=0, abs_tol=1e-10):
039 |             raise ValueError("Vm must lie between 0 and 1 and Vf + Vm must equal 1.")
040 |     # Use the exact complement after validating any supplied Vm.
041 |     return Vf, 1.0 - Vf
042 |
043 |
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Module documentation: validation is shared instead of reinvented in each model.
L2	Imports finite-number and tolerance utilities from Python's math library.
L3	Imports NumPy to recognize array dimensions and NumPy Boolean values.
L6	Defines the basic validator for a single finite numeric input and its error label.
L7	Rejects True/False and nonscalar arrays. Booleans otherwise behave like integers in Python and could be mistaken for material data.
L8	Reports that the input must be a finite real scalar rather than an array or flag.
L9	Begins a guarded conversion to a standard Python floating-point value.
L10	Converts a numeric scalar, or even a convertible numeric string, to float. This is type conversion, not unit conversion.
L11	Catches failures caused by unsuitable types, nonnumeric contents or conversion overflow.
L12	Raises a labeled ValueError while retaining the original exception as its cause.
L13	Detects NaN and positive/negative infinity after conversion.
L14	Rejects those nonfinite values before they propagate into equations.
L15	Returns the validated float to the caller.
L18	Defines the stricter check used for elastic moduli.
L19	First applies all finite-scalar checks, reusing the function above.
L20	Tests whether the value is zero or negative.
L21	Rejects it because positive moduli are required for the supported solid phases.
L22	Returns the positive float unchanged in magnitude and unit.
L25	Defines the Poisson-ratio validator.
L26	Checks that nu is a finite scalar first.
L27	Enforces the open stability interval for a 3D isotropic linear elastic solid.
L28	Explains the admissible interval. The endpoints themselves are not accepted.
L29	Returns the admissible dimensionless Poisson ratio.
L32	Defines volume-fraction validation; matrix fraction is optional.
L33	Validates the reinforcement fraction as one finite scalar.
L34	Requires Vf to be between 0 and 1 inclusive.
L35	Makes explicit that percentages such as 30 must be supplied as 0.30.
L36	Checks whether the caller provided Vm rather than leaving it absent.
L37	Validates the supplied Vm as a finite scalar.

Source line(s)	Explanation and connection to the project
L38	Checks Vm's range and Vf+Vm=1 within an absolute 1e-10 tolerance. rel_tol=0 prevents scale-dependent tolerance.
L39	Rejects contradictory or physically impossible fractions.
L40	Comment explaining why the returned Vm is recomputed even when it was supplied.
L41	Returns Vf and its exact floating-point complement. This keeps all downstream models on one consistent two-phase fraction pair.

utilities/elastic_properties.py

Source file: `utilities/elastic_properties.py`

Theory and implementation

This module changes the representation of the same isotropic material, rather than predicting a new composite. `to_bulk_shear` maps E,nu to K,G; `properties` maps K,G to the dictionary used by prediction functions. `phases` prepares both constituents. `zeta` is the host-dependent shear polarization parameter used for spherical inclusions; it has modulus units and is not another material phase or an empirical fitting constant.

For the default inputs the phase values in GPa are: `Kf=38.888889`, `Gf=29.166667`, `Km=3.888889`, `Gm=1.296296`. The ratio forms in the inverse conversion are algebraic rearrangements of standard equations, chosen to avoid unnecessary large dimensional products.

Numbered source

```
001 | """Isotropic elastic conversions, phase preparation and sphere parameter."""
002 | from .input_checks import positive, poisson, fractions
003 |
004 |
005 | def to_bulk_shear(E, nu):
006 |     """K = E/[3(1-2nu)] and G = E/[2(1+nu)]."""
007 |     E, nu = positive(E, "E"), poisson(nu, "nu")
008 |     K, G = E / (3 * (1 - 2 * nu)), E / (2 * (1 + nu))
009 |     return positive(K, "K"), positive(G, "G")
010 |
011 |
012 | def properties(K, G):
013 |     """Return E, nu, K, G for a stable isotropic material."""
014 |     K, G = positive(K, "effective K"), positive(G, "effective G")
015 |     # Ratio form avoids unnecessary products of two dimensional moduli.
016 |     r = G / K
017 |     E = 9 * G / (3 + r)
018 |     nu = (3 - 2 * r) / (2 * (3 + r))
019 |     return {"E": positive(E, "effective E"), "nu": nu, "K": K, "G": G}
020 |
021 |
022 | def phases(Ef, Em, Vf, nu_f, nu_m, Vm=None):
023 |     Vf, Vm = fractions(Vf, Vm)
024 |     Kf, Gf = to_bulk_shear(Ef, nu_f)
025 |     Km, Gm = to_bulk_shear(Em, nu_m)
026 |     return Kf, Gf, Km, Gm, Vf, Vm
027 |
028 |
029 | def zeta(K, G):
030 |     """Shear polarization parameter for a sphere in an isotropic host."""
031 |     r = K / G
032 |     return G * (9 * r + 8) / (6 * (r + 2))
033 |
034 |
035 |
036 |
037 | def from_bulk_shear(K, G):
038 |     """Public inverse conversion: returns E, nu, K, G."""
039 |     return properties(K, G)
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Documents the three jobs of this module: conversions, input preparation and a spherical-host parameter.
L2	Imports scalar/modulus/Poisson/fraction checks from the sibling validator module.
L5	Defines a conversion that accepts one isotropic Young's modulus and Poisson ratio.
L6	Documents the equations evaluated below.
L7	Validates positive E and a stable nu; tuple assignment replaces both inputs with validated floats.
L8	Computes the bulk and shear moduli using isotropic elasticity. Both retain E's unit.
L9	Checks that the calculated K and G are finite and positive, then returns the pair.
L12	Defines the inverse conversion and the standard elastic-result dictionary.
L13	Documents the four returned properties.
L14	Checks both effective moduli before dividing or converting them.
L15	Comment giving the numerical reason for the following algebraic rearrangement.
L16	Forms the dimensionless ratio r=G/K.
L17	Computes E=9G/(3+G/K), which equals 9KG/(3K+G).
L18	Computes nu=(3-2G/K)/[2(3+G/K)], the standard inverse conversion.
L19	Returns named values. E is checked again; nu is dimensionless, while E,K,G share one unit.
L22	Defines a common preparation step for models receiving both phases and fractions.
L23	Checks Vf and optional Vm and computes the consistent complementary pair.
L24	Converts reinforcement E and nu into Kf and Gf.
L25	Converts matrix E and nu into Km and Gm.
L26	Returns the six validated quantities in a fixed unpacking order.
L29	Defines the spherical host's shear-polarization parameter zeta.
L30	Documents that the parameter is specific to a sphere in an isotropic host.
L31	Forms K/G to express the parameter without unnecessary K*G products.
L32	Evaluates zeta=G(9K+8G)/[6(K+2G)] in an equivalent ratio form. Callers supply positive host moduli.
L37	Defines the public inverse-conversion name used by tests or user scripts.
L38	Documents the same four-property result returned by properties.
L39	Delegates to properties so the inverse equations and checks have one implementation.

utilities/tensor_operations.py

Source file: `utilities/tensor_operations.py`

Theory and implementation

Read the Kelvin explanation in section 4 first. The matrices encode a constitutive law, not the shape of a mesh. No finite elements are created. `projectors` returns J and D; `stiffness` assembles C; the extractor recovers K,G and then E,nu from an isotropic effective C. The extraction uses elementwise multiplication followed by a sum to implement a matrix inner product.

Numbered source

```
001 | """Orthonormal Kelvin tensor operations; not engineering Voigt notation."""
002 | import numpy as np
003 | from .elastic_properties import properties
004 |
005 |
006 | def projectors():
007 |     """Hydrostatic J and deviatoric D in orthonormal Kelvin notation.
008 |
009 |     Vector order: [11, 22, 33, sqrt(2)*23, sqrt(2)*13, sqrt(2)*12].
010 |     The same scaling applies to stress and strain, so ordinary matrix
011 |     multiplication represents fourth-order tensor contraction.
012 |     """
013 |     h = np.array([1, 1, 1, 0, 0, 0], dtype=float) / np.sqrt(3)
014 |     J = np.outer(h, h)
015 |     return J, np.eye(6) - J
016 |
017 |
018 | def stiffness(K, G):
019 |     J, D = projectors()
020 |     return 3 * K * J + 2 * G * D
021 |
022 |
023 | def properties_from_stiffness(C):
024 |     """Extract isotropic K and G from a Kelvin stiffness matrix."""
025 |     J, D = projectors()
026 |     K = float(np.sum(J * C) / 3)
027 |     G = float(np.sum(D * C) / 10)
028 |     return properties(K, G)
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Declares the Kelvin convention so the matrices are not mistaken for engineering-Voigt matrices.
L2	Imports NumPy arrays and matrix operations.
L3	Imports conversion of effective K,G to the standard result dictionary.
L6	Defines construction of hydrostatic and deviatoric projection matrices.
L7-12	Docstring states the vector order and equal stress/strain shear scaling, which make ordinary matrix multiplication represent tensor contraction.
L13	Builds the unit hydrostatic direction h. Dividing by sqrt(3) makes its squared length equal to one.
L14	Forms the outer product h h^T. J keeps the hydrostatic part of a strain or stress vector.
L15	Returns J and D=I-J; D removes the hydrostatic part and keeps the shape-changing part.
L18	Defines stiffness assembly from an isotropic pair K,G.
L19	Obtains both projectors using the common tensor convention.
L20	Builds C=3KJ+2GD. Hydrostatic strain is scaled by 3K and deviatoric strain by 2G.
L23	Defines extraction from an isotropic Kelvin stiffness matrix.
L24	Documents the intended input symmetry and representation; arbitrary anisotropic inputs are not validated here.
L25	Obtains the same projectors used in assembly.
L26	Computes the Frobenius contraction J:C and divides by 3 because J:C=3K. float converts the NumPy scalar to a Python float.
L27	Computes D:C and divides by 10 because D:D=5 and the deviatoric eigenvalue is 2G.
L28	Converts the recovered effective K,G into E,nu,K,G with the shared positivity checks.

models/_common.py

Source file: `models/_common.py`

Why this extra file exists

This four-line compatibility module was added beyond your original tree. The first model-only delivery stored shared helper functions here. The complete project moved their implementation into `utilities`, while retaining the same model import statements. There are no duplicate equations or new calculations in this file. The complete models and utilities folders must stay together.

Numbered source

```
001 | """Compatibility imports for the model modules. Implementations live in utilities."""
002 | from utilities.input_checks import finite_scalar, positive, poisson, fractions
003 | from utilities.elastic_properties import to_bulk_shear, properties, phases, zeta
004 | from utilities.tensor_operations import projectors, stiffness, properties_from_stiffness
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Documents that this file re-exports definitions rather than implementing them.
L2	Makes the four validation helpers available to models through their existing ._common imports.
L3	Re-exports conversion, phase preparation, result formatting and the sphere host parameter.
L4	Re-exports the Kelvin projectors, stiffness assembly and isotropic extraction helpers.

models/rule_of_mixtures.py

Source file: `models/rule_of_mixtures.py`

Deriving the two scalar baselines

If both phases have the same axial strain e, average axial stress is `Vf Ef e + Vm Em e`; dividing by e gives the arithmetic E mixture. If the phases have the same axial stress s, average strain is `Vf s/Ef + Vm s/Em`; dividing s by that strain gives the harmonic mixture. These are simplified 1D mechanical idealizations, comparable to parallel and series springs. They do not specify a full 3D composite microstructure.

The extra `isotropic_voigt_reuss` function performs full isotropic stiffness and compliance averaging via K and G. Its upper E need not equal the simple arithmetic E mixture when constituent Poisson ratios differ. For the default case at Vf=.05, the simple arithmetic E is 6.825 GPa, while the proper isotropic Voigt E bound is 6.962397 GPa. These are different calculations by design. The nu values in bounding dictionaries are not independent bounds on nu.

Numbered source

```
001 | """Simple E mixture formulas and separate 3-D isotropic elastic bounds.
002 |
003 | The arithmetic mixture of E is a 1-D equal-strain idealization. When phase
004 | Poisson ratios differ it is not the rigorous 3-D Voigt upper bound on E.
005 | Use isotropic_voigt_reuss for the full isotropic stiffness/compliance bounds.
006 |
007 | Reference: https://jfbarthelemy.github.io/echoes/99_appendices/scheme_derivations.html
008 | """
009 |
010 | from ._common import fractions, phases, positive, properties
011 |
012 |
013 | def voigt(Ef, Em, Vf, Vm=None):
014 |     """Return the simple equal-strain estimate Vf*Ef + Vm*Em."""
015 |     Ef, Em = positive(Ef, "Ef"), positive(Em, "Em")
016 |     Vf, Vm = fractions(Vf, Vm)
017 |     return Vf * Ef + Vm * Em
018 |
019 |
020 | def reuss(Ef, Em, Vf, Vm=None):
021 |     """Return the simple equal-stress estimate 1/(Vf/Ef + Vm/Em)."""
022 |     Ef, Em = positive(Ef, "Ef"), positive(Em, "Em")
023 |     Vf, Vm = fractions(Vf, Vm)
024 |     return 1 / (Vf / Ef + Vm / Em)
025 |
026 |
027 | def rule_of_mixtures(Ef, Em, Vf, Vm=None):
028 |     """Return both 1-D E estimates, in the same units as Ef and Em."""
029 |     return {"E_voigt": voigt(Ef, Em, Vf, Vm), "E_reuss": reuss(Ef, Em, Vf, Vm)}
030 |
031 |
032 | def isotropic_voigt_reuss(Ef, Em, Vf, *, nu_f, nu_m, Vm=None):
033 |     """Return rigorous isotropic lower/upper elastic-property bounds.
034 |
035 |     Average K and G arithmetically for Voigt, harmonically for Reuss,
036 |     then convert each pair to E and nu. The returned nu values are those
037 |     of the bounding isotropic tensors, not separate bounds on nu.
038 |     """
039 |     Kf, Gf, Km, Gm, Vf, Vm = phases(Ef, Em, Vf, nu_f, nu_m, Vm)
040 |     return {
041 |         "lower": properties(1 / (Vf / Kf + Vm / Km), 1 / (Vf / Gf + Vm / Gm)),
042 |         "upper": properties(Vf * Kf + Vm * Km, Vf * Gf + Vm * Gm),
043 |     }
```

Line commentary

Source line(s)	Explanation and connection to the project
L1-8	Module docstring distinguishes 1D scalar E mixtures from 3D isotropic bounds and records a theory reference.
L10	Imports validation, phase conversion and result-formatting helpers.
L13	Defines the equal-strain scalar estimate; Vm can be supplied or inferred.
L14	Documents the arithmetic mixture equation.
L15	Validates positive constituent moduli without changing units.
L16	Validates fractions and obtains the consistent matrix complement.
L17	Computes the phase-volume-weighted arithmetic average of E.
L20	Defines the equal-stress scalar estimate with the same input interface.
L21	Documents the harmonic mixture equation.
L22	Checks both moduli before taking their reciprocals.
L23	Checks the two-phase volume constraint.
L24	Averages axial compliances 1/E and takes the reciprocal, giving the equal-stress effective E.
L27	Defines a convenience function returning both scalar estimates in one call.
L28	Documents that both results retain the constituent modulus unit.
L29	Calls the validated scalar functions and labels their results E_voigt and E_reuss.
L32	Defines the separate 3D isotropic bounds; keyword-only nu_f and nu_m make the extra required inputs explicit.
L33-38	Docstring explains K/G averaging and warns against interpreting the two returned nu values as nu bounds.
L39	Validates and converts both phases to K,G and prepares the fractions.
L40	Starts a nested dictionary whose lower and upper entries each hold four elastic properties.
L41	Harmonically averages K and G for the Reuss compliance bound, then converts the pair to E and nu.
L42	Arithmetically averages K and G for the Voigt stiffness bound, then converts the pair to E and nu.
L43	Closes and returns the nested bounds dictionary.

models/eshelby.py

Source file: `models/eshelby.py`

Theory: constraint and concentration are different

Imagine a spherical region that would undergo a stress-free transformation strain e* if left alone. Its surrounding matrix constrains that deformation. Eshelby's tensor maps e* to the constrained inclusion strain in that auxiliary problem. In the spherical isotropic case its Cartesian components are:

```
S_ijkl = a delta_ij delta_kl + b(delta_ik delta_jl + delta_il delta_jk)
a = (5nu_m-1)/[15(1-nu_m)]
b = (4-5nu_m)/[15(1-nu_m)]
S = [(1+nu_m)/(3(1-nu_m))] J + 2b D       (Kelvin form)
```

`delta` is the Kronecker delta: one if two indices agree, zero otherwise. S is dimensionless. For this shape it depends only on host nu_m. The phase stiffness contrast enters a different tensor:

```
DeltaC = Cf-Cm
A_dil = [I + S Cm^-1 DeltaC]^-1
C_dil = Cm + Vf DeltaC A_dil
```

A_dil maps the undisturbed far-field matrix strain to inclusion strain. For a stiffer inclusion it is generally below one in the relevant strain subspaces: that inclusion deforms less under the same remote loading. The dilute effective estimate assumes inclusions do not interact. The chosen Vf=.1 display cutoff is a teaching policy, not a universal physics threshold.

Unpack line 56 carefully: first solve Cm X=DeltaC, then form M=I+S X, then solve M A=I. `solve` implements the needed inverse actions without explicitly writing `inv(Cm)` or `inv(M)`. A fourth-order elasticity tensor is represented by a 6x6 matrix here; it is not being replaced with a scalar.

Numbered source

```
001 | """Spherical Eshelby tensor and its dilute effective-stiffness estimate.
002 |
003 | Assumptions: small strain, linear elasticity, isotropic phases, perfect
004 | bonding, spherical inclusions. Cylindrical fibers and woven architecture
005 | are not implemented here. The dilute approximation neglects inclusion
006 | interactions; it is not a valid full-volume-fraction prediction.
007 |
008 | References:
009 | Eshelby (1957), doi:10.1098/rspa.1957.0133.
010 | https://jfbarthelemy.github.io/echoes/99_appendices/scheme_derivations.html
011 | """
012 |
013 | import warnings
014 |
015 | import numpy as np
016 |
017 | from ._common import (
018 |     fractions, phases, poisson, projectors, properties_from_stiffness,
019 |     stiffness, to_bulk_shear,
020 | )
021 |
022 |
023 | def sphere_eshelby(nu_m, *, notation="kelvin"):
024 |     """Return dimensionless S for a sphere in an infinite isotropic host.
025 |
026 |     'kelvin' returns shape (6,6), using sqrt(2) shear scaling, NOT
027 |     engineering Voigt notation. 'cartesian' returns S[i,j,k,l].
028 |     S maps transformation eigenstrain to constrained inclusion strain;
029 |     it is distinct from the inhomogeneity strain-concentration tensor.
030 |     """
031 |     nu_m = poisson(nu_m, "nu_m")
032 |     a = (5 * nu_m - 1) / (15 * (1 - nu_m))
033 |     b = (4 - 5 * nu_m) / (15 * (1 - nu_m))
```

```
    if notation == "kelvin":
        J, D = projectors()
        return ((1 + nu_m) / (3 * (1 - nu_m))) * J + 2 * b * D
    if notation == "cartesian":
        delta = np.eye(3)
        return (
            a * np.einsum("ij,kl->ijkl", delta, delta)
            + b * np.einsum("ik,jl->ijkl", delta, delta)
            + b * np.einsum("il,jk->ijkl", delta, delta)
        )
    raise ValueError("notation must be 'kelvin' or 'cartesian'.")


def dilute_concentration(Ef, Em, *, nu_f, nu_m):
    """Return Kelvin A = [I + S Cm^-1 (Cf-Cm)]^-1.

    A maps the undisturbed far-field matrix strain to inclusion strain.
    """
    Kf, Gf = to_bulk_shear(Ef, nu_f)
    Km, Gm = to_bulk_shear(Em, nu_m)
    Cf, Cm = stiffness(Kf, Gf), stiffness(Km, Gm)
    S = sphere_eshelby(nu_m)
    return np.linalg.solve(np.eye(6) + S @ np.linalg.solve(Cm, Cf - Cm), np.eye(6))


def eshelby_dilute(Ef, Em, Vf, *, nu_f, nu_m, Vm=None):
    """Return E, nu, K, G from C_dil = Cm + Vf*(Cf-Cm)*A.

    A warning above Vf=0.1 is a teaching reminder, not a universal
    validity threshold. Even below 0.1 accuracy depends on contrast and
    microstructure. Vf=1 is rejected: this approximation cannot recover
    the pure-inclusion limit in general. No endpoint is patched in.
    """
    Kf, Gf, Km, Gm, Vf, _ = phases(Ef, Em, Vf, nu_f, nu_m, Vm)
    if Vf == 1:
        raise ValueError("The dilute approximation is not valid at Vf=1.")
    if Vf > 0.1:
        warnings.warn(
            "Dilute estimate at Vf > 0.1: inclusion interactions are neglected; "
            "this is an extrapolation, not a validated prediction.",
            UserWarning, stacklevel=2,
        )
    Cf, Cm = stiffness(Kf, Gf), stiffness(Km, Gm)
    A = dilute_concentration(Ef, Em, nu_f=nu_f, nu_m=nu_m)
    return properties_from_stiffness(Cm + Vf * (Cf - Cm) @ A)
```

Line commentary

Source line(s)	Explanation and connection to the project
L1-11	Module documentation states spherical/isotropic/perfect-bond assumptions, the low-concentration limitation and references.
L13	Imports a warning mechanism so extrapolated dilute calls can alert the user without immediately stopping.
L15	Imports array, identity, tensor-product and linear-solve operations.
L17-20	Imports common preparation and tensor helpers. fractions is imported here but is not directly used; phases handles fraction validation.
L23	Defines S for a given host Poisson ratio, with keyword-only representation choice.
L24-30	Docstring distinguishes Kelvin versus Cartesian shape and clarifies that S is not A_dil.
L31	Checks that the matrix Poisson ratio is admissible.
L32	Calculates the coefficient multiplying delta_ij delta_kl in Cartesian S.
L33	Calculates the coefficient multiplying the two symmetric index-pair products.
L34	Selects the 6x6 orthonormal Kelvin representation.
L35	Obtains hydrostatic and deviatoric projectors.
L36	Assembles the hydrostatic and deviatoric eigenparts of S. Both coefficients are dimensionless.
L37	Selects the full 3x3x3x3 Cartesian representation instead.
L38	Builds the 3x3 identity, which stores delta_ij.
L39	Begins the sum of the three fourth-order arrays.
L40	Uses Einstein-index notation to build a delta_ij delta_kl term; each output entry has indices i,j,k,l.
L41	Adds the delta_ik delta_jl term with coefficient b.
L42	Adds the delta_il delta_jk term, ensuring symmetry in the strain/stress index pairs.
L43	Completes and returns the fourth-order tensor.
L44	Rejects unsupported notation strings instead of silently using a different convention.
L47	Defines concentration of the inclusion strain relative to undisturbed matrix strain.
L48-51	Docstring gives the concentration equation and states which reference strain is used.
L52	Converts the inclusion's E,nu to bulk and shear moduli.
L53	Converts the matrix's E,nu to bulk and shear moduli.
L54	Assembles the two isotropic Kelvin stiffness matrices in the same convention.
L55	Obtains the spherical S using the host's Poisson ratio.
L56	Solves Cm X=Cf-Cm, forms I+S@X, then solves that matrix times A=I. Returns dimensionless A_dil.
L59	Defines the low-concentration effective-property prediction.

Source line(s)	Explanation and connection to the project
L60-66	Documents the dilute stiffness formula, the non-universal warning threshold and why Vf=1 is not patched to the inclusion value.
L67	Validates phases and fractions; the final underscore ignores Vm because this dilute formula uses Vf explicitly.
L68	Checks the pure-inclusion endpoint.
L69	Rejects Vf=1, where the dilute approximation does not generally reproduce the physical limit.
L70	Checks whether a direct call is above the educational low-fraction interval.
L71-75	Emits a UserWarning describing extrapolation. stacklevel=2 points to the caller's line. The call can still return an extrapolated value.
L76	Constructs the phase stiffness matrices needed by C_dil.
L77	Computes the concentration tensor using the actual spherical Eshelby calculation.
L78	Evaluates C_dil=Cm+Vf*(Cf-Cm)@A and extracts E,nu,K,G. Scalar multiplication and matrix multiplication have distinct roles.

models/mori_tanaka.py

Source file: `models/mori_tanaka.py`

Deriving the normalization used in the code

Let e_m be average matrix strain. The dilute inclusion relation is used as `e_f = A_dil e_m`. Average composite strain is then `e_bar = Vm e_m + Vf e_f = (Vm I + Vf A_dil)e_m = B e_m`. Therefore `e_m=B^-1 e_bar` and `e_f=A_dil B^-1 e_bar`. The normalized inclusion factor is `Af=A_dil B^-1`.

Average stress can be rearranged into `Ceff = Cm + Vf(Cf-Cm)Af`. This is what the final line evaluates. The normalization enforces consistency of average strain and includes the matrix-mediated phase interaction assumed by Mori-Tanaka. It does not compute the exact interaction of every pair of particles.

To compute A B^-1 without explicitly constructing B^-1, solve `B^T X^T=A^T`; transposing the solution gives X=A B^-1. This explains the otherwise unusual transpose/solve/transpose expression on line 35.

Numbered source

```
001 | """Mori-Tanaka for isotropic phases with spherical inclusions.
002 |
003 | Uses the actual spherical Eshelby tensor, in orthonormal Kelvin notation.
004 | The matrix is the connected host. Ideal interfaces and linear elasticity
005 | are assumed. This is not a cylindrical-fiber or woven-fabric model.
006 |
007 | Reference: https://doi.org/10.1093/jge/gxab034 (spherical P,Q factors).
008 | """
009 |
010 | import numpy as np
011 |
012 | from ._common import phases, properties, properties_from_stiffness, stiffness
013 | from .eshelby import dilute_concentration
014 |
015 |
016 | def mori_tanaka(Ef, Em, Vf, *, nu_f, nu_m, Vm=None):
017 |     """Return E, nu, K, G; all dimensional values retain the input unit.
018 |
019 |     B = Vm*I + Vf*A_dil
020 |     A_f = A_dil * B^-1
021 |     C_eff = Cm + Vf*(Cf-Cm)*A_f
022 |
023 |     The Vf=1 limit is algebraically correct, although the connected-matrix
024 |     interpretation loses physical relevance when the matrix disappears.
025 |     """
026 |     Kf, Gf, Km, Gm, Vf, Vm = phases(Ef, Em, Vf, nu_f, nu_m, Vm)
027 |     if Vf == 0:
028 |         return properties(Km, Gm)
029 |     if Vf == 1:
030 |         return properties(Kf, Gf)
031 |     Cf, Cm = stiffness(Kf, Gf), stiffness(Km, Gm)
032 |     A = dilute_concentration(Ef, Em, nu_f=nu_f, nu_m=nu_m)
033 |     B = Vm * np.eye(6) + Vf * A
034 |     # Solve for A @ inv(B) without explicitly forming an inverse.
035 |     Af = np.linalg.solve(B.T, A.T).T
036 |     return properties_from_stiffness(Cm + Vf * (Cf - Cm) @ Af)
```

Line commentary

Source line(s)	Explanation and connection to the project
L1-8	Documents the matrix-host assumption, spherical scope, tensor convention and formula source.
L10	Imports identities and stable linear-system solves.
L12	Imports phase preparation, endpoint property conversion and stiffness/tensor extraction helpers.
L13	Imports the inclusion concentration tensor from the Eshelby module, creating the theory-to-code dependency.
L16	Defines the public Mori-Tanaka predictor with explicit phase Poisson ratios.
L17-25	Documents B, Af and Ceff and explains the mathematical versus physical meaning of the Vf=1 limit.
L26	Validates the two phases and obtains Kf,Gf,Km,Gm,Vf,Vm.
L27	Tests whether there is no reinforcement.
L28	Returns the matrix properties exactly at that limit.
L29	Tests the pure-inclusion mathematical endpoint.
L30	Returns the inclusion properties there; this is not proof that a connected matrix exists at Vf=1.
L31	Builds the two Kelvin stiffness matrices.
L32	Computes A_dil through the spherical Eshelby tensor and phase stiffness contrast.
L33	Forms B=Vm I+Vf A, the map from average matrix strain to macroscopic strain.
L34	Comment explaining the avoidance of an explicit inverse.
L35	Uses a transposed linear solve to calculate Af=A@inverse(B), the inclusion concentration relative to macroscopic strain.
L36	Assembles the effective stiffness and returns its isotropic elastic properties, including the target E.

models/self_consistent.py

Source file: `models/self_consistent.py`

Theory and the reason an iteration is needed

Each phase is represented as a spherical inclusion in the unknown effective medium itself. Its K and G enter the host response, so they occur on both sides of the equations. This differs from assuming a known matrix host.

```
rK = sum_i Vi (Ki-K)/(Ki+4G/3) = 0
rG = sum_i Vi (Gi-G)/(Gi+zeta(K,G)) = 0
```

This implementation reduces the two-variable problem without changing those equations. For a fixed positive G define `wi=Vi/(Ki+4G/3)`. The first equation is `sum(wi Ki)-K sum(wi)=0`, giving `K=sum(wi Ki)/sum(wi)`. Thus the code can compute K exactly for every trial G and solve just the remaining shear residual.

The search uses `log(G)`. A midpoint in logarithmic space corresponds to a geometric mean of the positive shear endpoints; positive trial values are guaranteed. Dimensionless scaling keeps the residual calculation independent of whether you entered Pa, MPa or GPa. Bisection updates the bracket according to the sign of rG. A returned solution must satisfy both residual tolerances.

This is numerical solution of analytical effective-medium equations. It does not use a finite-element mesh. `tolerance=1e-9` is a bound on dimensionless equation residuals, not an experimentally established error in predicted E.

Numbered source

```
001 | """Classical self-consistent scheme for two isotropic solid phases.
002 |
003 | Both phases are modeled as spheres in the unknown isotropic effective
004 | medium. This is the classical scheme, not the three-phase generalized
005 | self-consistent coated-sphere model. Numerical solution of analytical
006 | equations does not require a finite-element simulation.
007 |
008 | References:
009 | https://archive.nptel.ac.in/content/storage2/courses/101104010/lecture34/34_2.htm
010 | https://jfbarthelemy.github.io/echoes/99_appendices/scheme_derivations.html
011 | """
012 |
013 | import math
014 |
015 | import numpy as np
016 |
017 | from ._common import finite_scalar, phases, properties, zeta
018 |
019 |
020 | def self_consistent(Ef, Em, Vf, *, nu_f, nu_m, Vm=None,
021 |                     tolerance=1e-9, max_iterations=500):
022 |     """Return E, nu, K, G and solver diagnostics.
023 |
024 |     Solve simultaneously:
025 |       sum_i Vi*(Ki-K)/(Ki+4G/3) = 0
026 |       sum_i Vi*(Gi-G)/(Gi+zeta(K,G)) = 0
027 |
028 |     For any positive trial G, the bulk equation gives K explicitly as a
029 |     weighted harmonic mean of Ki+4G/3, minus 4G/3. A cancellation-free
030 |     equivalent is used below. The remaining shear equation is solved by
031 |     bisection in log(G), bracketed by the phase shear moduli. Both residuals
032 |     are checked. Only NumPy and the Python standard library are required.
033 |     """
034 |     Kf, Gf, Km, Gm, Vf, Vm = phases(Ef, Em, Vf, nu_f, nu_m, Vm)
035 |     tolerance = finite_scalar(tolerance, "tolerance")
036 |     if not 1e-12 <= tolerance <= 1e-4:
037 |         raise ValueError("tolerance must be between 1e-12 and 1e-4.")
038 |     if isinstance(max_iterations, (bool, np.bool_)) or not isinstance(
```

```
039 |         max_iterations, (int, np.integer)
040 |     ) or max_iterations < 1:
041 |         raise ValueError("max_iterations must be a positive integer.")
042 |
043 |     def endpoint(K, G):
044 |         return {**properties(K, G), "converged": True,
045 |                 "residual_norm": 0.0, "iterations": 0}
046 |
047 |     if Vf == 0:
048 |         return endpoint(Km, Gm)
049 |     if Vf == 1:
050 |         return endpoint(Kf, Gf)
051 |     if Kf == Km and Gf == Gm:
052 |         return endpoint(Km, Gm)
053 |
054 |     scale = max(Kf, Gf, Km, Gm)
055 |     Ks = np.array([Kf, Km]) / scale
056 |     Gs = np.array([Gf, Gm]) / scale
057 |     volumes = np.array([Vf, Vm])
058 |
059 |     def evaluate(G):
060 |         bulk_weights = volumes / (Ks + 4 * G / 3)
061 |         K = float(np.dot(bulk_weights, Ks) / np.sum(bulk_weights))
062 |         rK = float(np.sum(volumes * (Ks - K) / (Ks + 4 * G / 3)))
063 |         rG = float(np.sum(volumes * (Gs - G) / (Gs + zeta(K, G))))
064 |         return K, rK, rG
065 |
066 |     lower, upper = float(np.min(Gs)), float(np.max(Gs))
067 |     _, _, r_lower = evaluate(lower)
068 |     _, _, r_upper = evaluate(upper)
069 |     if r_lower < -tolerance or r_upper > tolerance:
070 |         raise RuntimeError("Could not bracket a positive self-consistent solution.")
071 |     lo, hi = math.log(lower), math.log(upper)
072 |     last_norm = float("inf")
073 |     for iteration in range(1, max_iterations + 1):
074 |         mid = (lo + hi) / 2
075 |         G = math.exp(mid)
076 |         K, rK, rG = evaluate(G)
077 |         last_norm = max(abs(rK), abs(rG))
078 |         if last_norm <= tolerance:
079 |             return {**properties(K * scale, G * scale), "converged": True,
080 |                     "residual_norm": last_norm,
081 |                     "iterations": iteration}
082 |         if rG > 0:
083 |             lo = mid
084 |         else:
085 |             hi = mid
086 |     raise RuntimeError(
087 |         f"Self-consistent solver did not converge; residual={last_norm:.3g}. "
088 |         "Check the phase properties or increase max_iterations."
089 |     )
```

Line commentary

Source line(s)	Explanation and connection to the project
L1-11	Documentation distinguishes the classical spherical self-consistent model from generalized coated-sphere schemes and supplies references.
L13	Imports log, exp and other scalar mathematical functions.
L15	Imports phase arrays, reductions and NumPy scalar-type checks.
L17	Imports input preparation, scalar tolerance validation, result conversion and the sphere parameter.
L20-21	Defines the predictor; defaults are a 1e-9 residual tolerance and at most 500 bisection iterations.
L22-33	Docstring states both governing residuals and the scalar-reduction strategy, making the solver reproducible.
L34	Validates and converts phases and volume fractions.
L35	Requires a finite scalar tolerance instead of accepting a list, NaN or Boolean.
L36	Restricts requested tolerance to the implemented meaningful numeric interval.
L37	Rejects values outside that interval with a readable message.
L38-40	Checks that max_iterations is a positive Python or NumPy integer, explicitly excluding Boolean flags.
L41	Rejects an invalid iteration budget before entering the solver.
L43	Defines a local helper for cases with an exact known answer.
L44-45	Combines E,nu,K,G with successful diagnostics. Exact endpoints have zero residual and need no iterations.
L47	Tests the pure matrix case.
L48	Returns the exact matrix solution via the diagnostic helper.
L49	Tests the pure inclusion case.
L50	Returns the exact inclusion solution.
L51	Checks whether both constituent bulk and shear moduli are identical.
L52	Returns the common phase without solving an unnecessary nonlinear problem.
L54	Chooses the largest phase modulus as a positive dimensional reference scale.
L55	Stores the two phase bulk moduli divided by the scale, making them dimensionless.
L56	Stores the two scaled phase shear moduli in matching phase order.
L57	Stores Vf and Vm as weights aligned with the K and G phase arrays.
L59	Defines a local evaluation function for one positive, scaled trial shear modulus.
L60	Calculates wi=Vi/(Ki+4G/3), the weights obtained by rearranging the bulk residual.
L61	Calculates the K that satisfies the bulk equation for this trial G, using the weighted mean derived above.
L62	Re-evaluates the bulk residual explicitly instead of assuming algebraic elimination is numerically exact.

Source line(s)	Explanation and connection to the project
L63	Evaluates the shear residual with the unknown effective K,G used as the host properties.
L64	Returns the trial effective K and both dimensionless residuals.
L66	Brackets the effective shear modulus between the two positive phase shear moduli.
L67	Evaluates the shear residual at the lower endpoint; underscores discard the unused K and rK.
L68	Evaluates it at the upper endpoint.
L69	Checks that the endpoint signs are consistent with the positive-to-negative bisection bracket, allowing tolerance-scale roundoff.
L70	Stops if a usable bracket was not found rather than returning an arbitrary estimate.
L71	Transforms both positive endpoints to logarithmic search coordinates.
L72	Initializes the most recent residual norm as infinity so failure reporting is always defined.
L73	Begins the bounded iteration loop, counting from one.
L74	Bisects the current bracket in log coordinates.
L75	Exponentiates to obtain a strictly positive scaled trial shear modulus.
L76	Computes the corresponding bulk modulus and both residuals.
L77	Defines the residual norm as the larger absolute equation error, so neither equation can be ignored.
L78	Tests the convergence criterion against the user's requested tolerance.
L79	Rescales K and G to the original modulus unit, converts them to elastic properties, and marks convergence.
L80	Stores the actual maximum dimensionless residual in the result.
L81	Stores how many bisection iterations were needed and completes the returned dictionary.
L82	Tests which half of the current bracket can contain the sign-changing shear root.
L83	For positive rG, raises the lower logarithmic endpoint to the trial midpoint.
L84	Handles a nonpositive shear residual when the tolerance test has not already passed.
L85	Lowers the upper endpoint to the midpoint, preserving the root bracket.
L86-89	Raises a solver failure after exhausting the iteration limit and reports the latest residual. No unconverged result is silently returned.

models/hashin_shtrikman.py

Source file: models/hashin_shtrikman.py

Theory: a range rather than a single best estimate

For a statistically isotropic effective material with suitable isotropic phase ordering, these bounds restrict the possible K and G for the given fractions. They do not require your material to have the spherical morphology used to derive the implemented mean-field predictions. In this implementation one phase must have both K>= and G>= the other. Ordering by E alone is not enough.

The code uses the contrast form of the bound formula instead of dividing directly by K_i-K_h or G_i-G_h. This avoids singular expressions when phase bulk or shear values happen to be equal. A softer reference phase gives the lower bound, a stiffer reference gives the upper bound. Converting the bounding K/G pairs gives bounding E values because E increases with positive K and G. The nu values of those pairs are not standalone nu bounds.

For ordered isotropic spherical phases, Mori-Tanaka follows the HS lower boundary for a soft matrix and the upper boundary for a stiff matrix. That equality is checked by tests; six model families need not give six distinct curves. See the project references for the bound formulas.

Numbered source

```
001 | """Hashin-Shtrikman bounds for well-ordered isotropic solid phases.
002 |
003 | These bound an isotropic effective composite, without specifying spherical
004 | microstructure. They must not be applied directly to a directional modulus
005 | of an aligned-fiber composite. Phases must be ordered in BOTH K and G;
006 | ordering only by Young's modulus is insufficient.
007 |
008 | Reference: https://rockphypy.readthedocs.io/en/latest/_modules/rockphypy/EM.html
009 | """
010 |
011 | from ._common import phases, properties, zeta
012 |
013 |
014 | def hashin_shtrikman(Ef, Em, Vf, *, nu_f, nu_m, Vm=None):
015 |     """Return {'lower': {E,nu,K,G}, 'upper': {E,nu,K,G}}.
016 |
017 |     For host h and other phase i, use the nonsingular form
018 |       K = Kh + vi*(Ki-Kh)/[1+vh*(Ki-Kh)/(Kh+4Gh/3)]
019 |       G = Gh + vi*(Gi-Gh)/[1+vh*(Gi-Gh)/(Gh+zeta_h)].
020 |     Soft host gives the lower bound; stiff host gives the upper bound.
021 |     The nu values describe the bounding tensors, not bounds on nu.
022 |     """
023 |     Kf, Gf, Km, Gm, Vf, Vm = phases(Ef, Em, Vf, nu_f, nu_m, Vm)
024 |     if Vf == 0 or Vf == 1:
025 |         p = properties(Km, Gm) if Vf == 0 else properties(Kf, Gf)
026 |         return {"lower": p.copy(), "upper": p.copy()}
027 |     if Kf >= Km and Gf >= Gm:
028 |         soft, stiff = (Km, Gm, Vm), (Kf, Gf, Vf)
029 |     elif Km >= Kf and Gm >= Gf:
030 |         soft, stiff = (Kf, Gf, Vf), (Km, Gm, Vm)
031 |     else:
032 |         raise ValueError(
033 |             "This implementation requires well-ordered phases: one phase "
034 |             "must have both K >= and G >= the other. Cross-ordered phases "
035 |             "require more general bounds."
036 |         )
037 |
038 |     def host_bound(host, other):
039 |         Kh, Gh, vh = host
040 |         Ki, Gi, vi = other
```

```
041 |         dK, dG = Ki - Kh, Gi - Gh
042 |         K = Kh + vi * dK / (1 + vh * dK / (Kh + 4 * Gh / 3))
043 |         G = Gh + vi * dG / (1 + vh * dG / (Gh + zeta(Kh, Gh)))
044 |         return properties(K, G)
045 |
046 |     return {"lower": host_bound(soft, stiff), "upper": host_bound(stiff, soft)}
```

Line commentary

Source line(s)	Explanation and connection to the project
L1-9	Documents effective isotropy, the phase-ordering restriction and the independent formula reference.
L11	Imports phase conversion, elastic-result conversion and the host shear parameter.
L14	Defines the bounds function with the same phase-input style as the predictors.
L15-22	Docstring specifies nested lower/upper outputs, contrast-form equations and the interpretation of nu.
L23	Validates the phases and converts them to K and G before deciding which is stiffer.
L24	Tests whether only one phase remains.
L25	Selects the present phase's properties for the pure matrix or pure inclusion case.
L26	Returns equal lower/upper values; separate copies prevent later mutation of one bound from changing the other.
L27	Checks whether the inclusion is at least as stiff in both bulk and shear response.
L28	Labels the matrix as soft and the inclusion as stiff, retaining each phase's own fraction.
L29	Checks the reverse ordering when the inclusion is not the stiffer phase in both quantities.
L30	Labels the inclusion as soft and the matrix as stiff.
L31	Handles cross-ordered phases, where neither of the two supported reference orderings applies.
L32-36	Rejects that case explicitly and explains that more general bounds are required. The comparison layer may record this as unavailable.
L38	Defines the local formula for a chosen reference host and the other phase.
L39	Unpacks host bulk modulus, shear modulus and its volume fraction.
L40	Unpacks the corresponding properties and fraction of the other phase.
L41	Forms the bulk and shear stiffness contrasts relative to the host.
L42	Evaluates the bulk bound in contrast form, remaining finite when dK=0.
L43	Evaluates the shear bound using the host's zeta parameter, likewise avoiding a 1/dG singularity.
L44	Converts each bound's K,G to the full elastic-property dictionary.
L46	Evaluates with the soft reference for the lower bound and stiff reference for the upper bound, then labels both results.

models/halpin_tsai.py

Source file: `models/halpin_tsai.py`

Theory and practical interpretation

This is a semi-empirical scalar modulus relation. Stiffness contrast is summarized by eta, while xi encodes a geometry/loading/calibration choice:

```
eta = (Ef/Em-1)/(Ef/Em+xi)
E = Em(1+xi eta Vf)/(1-eta Vf)
```

Unlike the sphere tensor calculation, the function does not derive xi from the geometry or compute a complete effective stiffness tensor. The project requires xi explicitly and labels its shipped value as a demonstration. It is not valid to call xi=2 a universal natural-fiber or spherical-particle constant. With xi=0 the expression reduces to the simple harmonic E mixture; with identical phases eta=0 and E=Em. These are useful mathematical checks.

Numbered source

```
001 | """Scalar Halpin-Tsai semi-empirical Young's modulus estimate.
002 |
003 | xi is required explicitly: its value depends on reinforcement geometry,
004 | loading direction and the chosen calibration. There is no universal xi
005 | for all composites. In particular xi=2 must not silently be taken as an
006 | isotropic spherical-particle model. This module predicts E only, not a
007 | complete stiffness tensor or a rigorous bound.
008 |
009 | Reference:
010 | https://archive.nptel.ac.in/content/storage2/courses/101104010/lecture34/34_4.htm
011 | """
012 |
013 | from ._common import finite_scalar, fractions, positive
014 |
015 |
016 | def halpin_tsai(Ef, Em, Vf, *, xi, Vm=None):
017 |     """Return {'E': ..., 'eta': ..., 'xi': ...}.
018 |
019 |     eta = (Ef/Em - 1)/(Ef/Em + xi)
020 |     E = Em*(1 + xi*eta*Vf)/(1 - eta*Vf)
021 |
022 |     xi >= 0 is allowed. xi=0 reduces to the simple Reuss E formula.
023 |     Call separately for each scalar Vf when making a comparison plot.
024 |     """
025 |     Ef, Em = positive(Ef, "Ef"), positive(Em, "Em")
026 |     Vf, _ = fractions(Vf, Vm)
027 |     xi = finite_scalar(xi, "xi")
028 |     if xi < 0:
029 |         raise ValueError("xi must be nonnegative and justified for the case.")
030 |     ratio = Ef / Em
031 |     eta = (ratio - 1) / (ratio + xi)
032 |     if Vf == 0:
033 |         E = Em
034 |     elif Vf == 1:
035 |         E = Ef
036 |     else:
037 |         E = Em * (1 + xi * eta * Vf) / (1 - eta * Vf)
038 |     return {"E": positive(E, "effective E"), "eta": eta, "xi": xi}
```

Line commentary

Source line(s)	Explanation and connection to the project
L1-11	Documentation describes the semi-empirical scalar scope and warns that xi is geometry/loading-dependent rather than universal.
L13	Imports finite-scalar, positive-modulus and fraction checks.
L16	Defines the predictor; xi must be supplied as a named argument rather than silently assumed.
L17-24	Docstring states the returned fields, governing equation and useful xi=0 limit.
L25	Checks both constituent Young's moduli.
L26	Validates the fraction pair, discarding the returned Vm because the expression uses Vf explicitly.
L27	Requires a finite scalar xi.
L28	Tests whether xi is negative.
L29	Rejects that unsupported parameter choice and reminds the user that geometry must justify it.
L30	Forms the dimensionless constituent stiffness ratio Ef/Em.
L31	Calculates eta, which summarizes phase contrast for this xi.
L32	Checks the pure matrix limit.
L33	Returns Em exactly at that limit.
L34	Checks the pure inclusion limit.
L35	Returns Ef exactly at that limit, avoiding endpoint cancellation in the rational formula.
L36	Selects the mixed-phase calculation.
L37	Evaluates the Halpin-Tsai equation. The dimensionless rational factor scales Em, so E retains its unit.
L38	Checks the final E and returns it along with eta and xi for interpretation. No K,G or nu is inferred from this one scalar estimate.

models/__init__.py

Source file: `models/__init__.py`

Purpose

This file provides convenient public names for the models package. It contains no stiffness equation. The leading dot in an import means another module in the same package. `__all__` controls wildcard imports, not which functions are physically valid for a particular material.

Numbered source

```
001 | """Analytical composite-modulus models.
002 |
003 | Import from the project parent: ``from models import mori_tanaka``.
004 | Input names are Ef, Em, Vf, optional Vm, and keyword-only nu_f/nu_m/xi.
005 | See README.md for scope, dependencies and a runnable example.
006 | """
007 |
008 | from .rule_of_mixtures import (
```

```
009 |     voigt, reuss, rule_of_mixtures, isotropic_voigt_reuss,
010 | )
011 | from .eshelby import sphere_eshelby, dilute_concentration, eshelby_dilute
012 | from .mori_tanaka import mori_tanaka
013 | from .self_consistent import self_consistent
014 | from .hashin_shtrikman import hashin_shtrikman
015 | from .halpin_tsai import halpin_tsai
016 |
017 | __all__ = [
018 |     "voigt", "reuss", "rule_of_mixtures", "isotropic_voigt_reuss",
019 |     "sphere_eshelby", "dilute_concentration", "eshelby_dilute",
020 |     "mori_tanaka", "self_consistent", "hashin_shtrikman", "halpin_tsai",
021 | ]
```

Line commentary

Source line(s)	Explanation and connection to the project
L1-6	Package documentation explains the public import pattern and common input names.
L8-10	Re-exports the two scalar mixtures, their combined wrapper and the separate isotropic bounds function.
L11	Re-exports the spherical Eshelby tensor, inclusion concentration tensor and dilute effective estimate.
L12	Re-exports Mori-Tanaka so users do not need to import its file directly.
L13	Re-exports the classical self-consistent function.
L14	Re-exports the isotropic bounds function.
L15	Re-exports Halpin-Tsai.
L17	Begins the explicit list of public names for from models import *.
L18	Lists the mixture and isotropic-bounds interface names.
L19	Lists the Eshelby-related interface names.
L20	Lists the remaining prediction/bounds functions.
L21	Completes the public-name list. Calling a function, not listing its name, performs the calculation.

data/materials.json

Source file: `data/materials.json`

Purpose and how to use it

JSON is a data format, not Python. Quotes identify keys/strings; braces enclose objects; brackets enclose a list. This file has one common unit and three material cases. No Vf is stored here: a single-run Vf comes from the command line and a sweep generates many Vf values.

Every shipped case is hypothetical. `source_status` and `source_note` record that limitation, rather than inventing a literature source. To model an actual specimen, obtain consistent phase inputs and check whether its architecture fits the model. Changing `name` to 'jute/epoxy' does not make spherical isotropic assumptions appropriate or create experimental validation.

Numbered source

```
001 | {
002 |   "schema_version": 1,
003 |   "unit": "GPa",
004 |   "materials": [
005 |     {
006 |       "id": "stiff_inclusions",
007 |       "name": "Stiff spherical inclusions (illustrative)",
008 |       "Ef": 70.0,
009 |       "Em": 3.5,
010 |       "nu_f": 0.20,
011 |       "nu_m": 0.35,
012 |       "xi": 2.0,
013 |       "geometry": "spherical",
014 |       "source_status": "illustrative",
015 |       "source_note": "Hypothetical teaching inputs, not measured glass/epoxy data. xi=2 is an
        uncalibrated parameter demonstration and is not a spherical-model calibration."
016 |     },
017 |     {
018 |       "id": "moderate_contrast",
019 |       "name": "Moderate stiffness contrast (illustrative)",
020 |       "Ef": 30.0,
021 |       "Em": 3.0,
022 |       "nu_f": 0.30,
023 |       "nu_m": 0.35,
024 |       "xi": 2.0,
025 |       "geometry": "spherical",
026 |       "source_status": "illustrative",
027 |       "source_note": "Hypothetical sensitivity case, not a measured biocomposite dataset. Real
        natural fibers require geometry and anisotropic-property decisions. xi=2 is an uncalibrated
        demonstration."
028 |     },
029 |     {
030 |       "id": "soft_inclusions",
031 |       "name": "Soft spherical inclusions (illustrative)",
032 |       "Ef": 1.0,
033 |       "Em": 10.0,
034 |       "nu_f": 0.30,
035 |       "nu_m": 0.25,
036 |       "xi": 2.0,
037 |       "geometry": "spherical",
038 |       "source_status": "illustrative",
039 |       "source_note": "Hypothetical case to check softening and reversed phase ordering. The
        inclusion is a solid, not a void. xi=2 is an uncalibrated demonstration."
040 |     }
041 |   ]
042 | }
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Opens the root JSON object containing the schema, units and material list.
L2	Sets the schema version expected by the loader; this identifies the file format, not a physical property.
L3	Declares GPa for every Ef and Em entry. The loader checks the label, but does not detect secretly mixed units.
L4	Begins the list of material cases, each stored as its own object.
L5	Opens the first material case.
L6	Sets a unique identifier used for command selection and output filenames.
L7	Sets the human-readable label used in terminal output and figures.
L8	Assigns inclusion E=70 GPa for this illustrative case.
L9	Assigns matrix E=3.5 GPa, giving an E contrast of 20.
L10	Assigns inclusion Poisson ratio 0.20; it is dimensionless.
L11	Assigns matrix Poisson ratio 0.35.
L12	Supplies xi=2 for the separate Halpin-Tsai parameter demonstration; it is not a calibrated sphere parameter.
L13	Records the spherical geometry supported by the actual inclusion calculations.
L14	Identifies the numbers as illustrative rather than measured or sourced material values.
L15	Explains provenance and the xi limitation in words carried through the outputs.
L16	Closes the first case; the comma indicates another object follows.
L17	Opens the second case.
L18	Sets the command/output identifier for a moderate-contrast sensitivity case.
L19	Provides its descriptive plot/terminal label.
L20	Assigns inclusion E=30 GPa.
L21	Assigns matrix E=3 GPa, giving an E contrast of 10.
L22	Assigns inclusion nu=0.30.
L23	Assigns matrix nu=0.35.
L24	Uses the same explicit uncalibrated xi demonstration value so parameter behavior can be inspected separately.
L25	Keeps the inclusion geometry spherical for a consistent isotropic study.
L26	Again marks the input values as illustrative.
L27	Explicitly avoids claiming that these values describe a measured natural-fiber composite.
L28	Closes the second case.

Source line(s)	Explanation and connection to the project
L29	Opens the third case.
L30	Identifies the soft-solid-inclusion case.
L31	Provides its descriptive label.
L32	Assigns inclusion E=1 GPa, deliberately lower than matrix E.
L33	Assigns matrix E=10 GPa; this tests softening and reversed phase ordering.
L34	Assigns inclusion nu=0.30.
L35	Assigns matrix nu=0.25.
L36	Records the Halpin-Tsai demonstration parameter explicitly.
L37	Keeps the geometry spherical; a soft inclusion is still not a modeled void.
L38	Marks the soft-inclusion input numbers as hypothetical.
L39	Explains the purpose and confirms that the inclusion is a positive-stiffness solid.
L40	Closes the final case; no following comma is needed.
L41	Closes the list of material objects.
L42	Closes the complete JSON document.

utilities/materials.py

Source file: `utilities/materials.py`

Why this additional module matters

Your actual complete project includes this file in addition to the original tree. It turns raw JSON into validated `Material` objects and preserves source notes. A dataclass holds named fields. `frozen=True` prevents ordinary field reassignment after creation, reducing accidental changes between calculations. Python type hints describe intended types; the runtime validators, not the hints, actually check JSON values.

The `from_dict` factory performs those checks. Directly constructing Material does not automatically call that factory. Likewise `dataclasses.replace` in main.py creates a new record; the Halpin-Tsai call then validates the xi override. The source-note check requires text, but cannot verify that the citation is scientifically correct. That still needs your judgment.

Numbered source

```
001 | """Load documented isotropic material cases from JSON."""
002 | from dataclasses import asdict, dataclass
003 | import json
004 | from pathlib import Path
005 | import re
006 |
007 | from .input_checks import finite_scalar, positive, poisson
008 |
009 |
010 | @dataclass(frozen=True)
011 | class Material:
012 |     id: str
013 |     name: str
014 |     Ef: float
015 |     Em: float
016 |     nu_f: float
017 |     nu_m: float
018 |     xi: float
019 |     source_status: str
020 |     source_note: str
021 |     geometry: str = "spherical"
022 |     unit: str = "GPa"
023 |
024 |     @classmethod
025 |     def from_dict(cls, raw, unit="GPa"):
026 |         if not isinstance(raw, dict):
027 |             raise ValueError("Each material case must be a JSON object.")
028 |         required = {"id", "name", "Ef", "Em", "nu_f", "nu_m", "xi", "source_status", "source_note"}
029 |         missing = sorted(required - raw.keys())
030 |         unknown = sorted(raw.keys() - required - {"geometry"})
031 |         if missing or unknown:
032 |             raise ValueError(f"Material fields: missing={missing}, unknown={unknown}")
033 |         for key in ["id", "name", "source_status", "source_note"]:
034 |             if not isinstance(raw[key], str) or not raw[key].strip():
035 |                 raise ValueError(f"{key} must be a nonempty string.")
036 |         if not re.fullmatch(r"[a-zA-Z0-9_-]+", raw["id"]):
037 |             raise ValueError("Material id may contain only letters, numbers, _ and -.")
038 |         if raw.get("geometry", "spherical") != "spherical":
039 |             raise ValueError("Only spherical isotropic inclusion cases are implemented.")
040 |         if unit not in {"Pa", "MPa", "GPa"}:
041 |             raise ValueError("unit must be Pa, MPa or GPa; every modulus must use it.")
042 |         xi = finite_scalar(raw["xi"], "xi")
043 |         if xi < 0:
044 |             raise ValueError("xi must be nonnegative.")
045 |         return cls(
046 |             id=raw["id"], name=raw["name"],
047 |             Ef=positive(raw["Ef"], "Ef"), Em=positive(raw["Em"], "Em"),
048 |             nu_f=poisson(raw["nu_f"], "nu_f"), nu_m=poisson(raw["nu_m"], "nu_m"),
```

```
            xi=xi, source_status=raw["source_status"], source_note=raw["source_note"],
            geometry="spherical", unit=unit,
        )

    def inputs(self, Vf):
        return dict(Ef=self.Ef, Em=self.Em, Vf=Vf, nu_f=self.nu_f, nu_m=self.nu_m)

    def to_dict(self):
        return asdict(self)


def load_materials(path):
    with Path(path).open(encoding="utf-8") as stream:
        data = json.load(stream)
    if not isinstance(data, dict) or data.get("schema_version") != 1:
        raise ValueError("Material data must be an object with schema_version=1.")
    if set(data) != {"schema_version", "unit", "materials"}:
        raise ValueError("Material file must contain schema_version, unit and materials only.")
    if not isinstance(data["materials"], list) or not data["materials"]:
        raise ValueError("materials must be a nonempty list.")
    result = {}
    for raw in data["materials"]:
        material = Material.from_dict(raw, data["unit"])
        if material.id in result:
            raise ValueError(f"Duplicate material id: {material.id}")
        result[material.id] = material
    return result
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Documents loading and documentation of isotropic material cases.
L2	Imports dataclass generation and conversion of a dataclass back to a dictionary.
L3	Imports the JSON parser.
L4	Imports portable path handling.
L5	Imports regular expressions to validate filename-safe case identifiers.
L7	Imports the property and parameter validators.
L10	Decorates the class as an immutable-style data record; Python generates its initializer and other common methods.
L11	Defines the Material record used by the comparison functions.
L12	Declares a string case identifier; later validation checks its allowed characters.
L13	Declares the human-readable name.
L14	Declares reinforcement Young's modulus.
L15	Declares matrix Young's modulus.
L16	Declares reinforcement Poisson ratio.
L17	Declares matrix Poisson ratio.
L18	Declares the Halpin-Tsai parameter.
L19	Declares the provenance category, such as illustrative.
L20	Declares explanatory source/assumption text.
L21	Sets the default geometry to spherical; the loader rejects unsupported geometry.
L22	Sets GPa as the default unit label.
L24	Makes the following factory a class method, receiving the class as cls rather than an existing instance.
L25	Defines creation from a raw dictionary and the file-wide unit.
L26	Checks that a case is actually an object/dictionary.
L27	Rejects a list, scalar or other incorrect case representation.
L28	Lists every required material field explicitly.
L29	Finds required fields missing from the input using set difference.
L30	Finds unknown keys, allowing geometry as an optional field. This catches misspellings such as Ef_typo.
L31	Tests whether either schema problem exists.
L32	Reports both missing and unknown keys together.
L33	Loops through the four required text fields.

Source line(s)	Explanation and connection to the project
L34	Requires a real nonempty string after whitespace is stripped.
L35	Reports which metadata field is missing meaningful text.
L36	Checks that the identifier contains only letters, digits, underscores or hyphens, preventing it from becoming an arbitrary output path.
L37	Explains the allowed identifier characters.
L38	Requires the supported geometry, treating an omitted geometry as spherical.
L39	Rejects unsupported geometry rather than pretending a cylinder calculation was performed.
L40	Checks the declared common modulus-unit label.
L41	Rejects unsupported labels; this does not convert between the three supported units.
L42	Validates the scalar xi field.
L43	Checks its sign.
L44	Rejects negative xi.
L45	Begins construction of the validated Material record.
L46	Copies the identifier and descriptive name.
L47	Validates and stores both positive phase Young's moduli.
L48	Validates and stores both stable isotropic Poisson ratios.
L49	Stores the checked xi and the supplied source descriptions.
L50	Stores the explicitly supported geometry and shared unit label.
L51	Completes and returns the record.
L53	Defines a convenience method that prepares predictor arguments for a requested Vf.
L54	Returns the phase moduli, Poisson ratios and Vf under names matching the model interfaces. Vm is inferred later.
L56	Defines conversion back to plain serializable data.
L57	Returns all dataclass fields as a dictionary, used in JSON manifests and tests.
L60	Defines reading a material database from disk.
L61	Opens the supplied path using UTF-8 and a context manager that closes the file automatically.
L62	Parses JSON text into Python dictionaries and lists.
L63	Checks the root type and supported schema version.
L64	Reports an unsupported or missing schema version.
L65	Requires exactly the three top-level keys, catching accidental changes to the format.
L66	Explains the expected root structure.
L67	Checks that the material collection is a nonempty list.

Source line(s)	Explanation and connection to the project
L68	Rejects an empty/malformed database.
L69	Starts a mapping from unique case identifiers to validated records.
L70	Visits each raw material object.
L71	Builds a validated Material using the common unit from the root file.
L72	Checks whether another record already uses the same identifier.
L73	Rejects duplicate names that would otherwise overwrite entries and output files.
L74	Adds the case under its identifier for later selection.
L75	Returns the complete validated material database.

analysis/compare_models.py

Source file: `analysis/compare_models.py`

How the theory becomes a results table

This module is the coordinator between material inputs and the model functions. It does not introduce another homogenization equation. `evaluate` produces 10 rows for one fraction: two simple mixtures, two isotropic VR bounds, dilute, MT, SC, two HS bounds and one HT demonstration.

Ten rows represent six families plus bounds, not ten independent physical methods. Missing values are empty strings, so CSV export and plots can preserve the difference between 'unavailable' and a physical zero. Every row carries a status and interpretation note. Dilute is intentionally not evaluated above .1 by this workflow even though its direct function can warn and extrapolate.

The HS `ValueError` handler is broad: it catches any ValueError raised there. Inputs have already been validated by earlier model calls, so unsupported K/G ordering is its intended use. Unexpected non-ValueError failures propagate.

Numbered source

```
001 | """Evaluate each model under explicit scope rules and export tidy tables."""
002 | import csv
003 | from pathlib import Path
004 |
005 | import numpy as np
006 |
007 | from models import (
008 |     eshelby_dilute, halpin_tsai, hashin_shtrikman, isotropic_voigt_reuss,
009 |     mori_tanaka, rule_of_mixtures, self_consistent,
010 | )
011 | from utilities.input_checks import finite_scalar, fractions
012 |
013 |
014 | FIELDS = ["case", "unit", "Vf", "Vm", "model", "E", "nu", "K", "G",
015 |           "status", "note", "residual_norm", "iterations"]
016 |
017 |
018 | def evaluate(material, Vf):
019 |     """Return one row per estimate/bound; unavailable results remain blank.
020 |
021 |     Dilute is evaluated only through Vf=0.1 as an educational restriction,
022 |     not a universal physical threshold. Cross-ordered HS bounds are skipped
023 |     with a reason; other calculation errors propagate and stop the run.
024 |     """
025 |     Vf, Vm = fractions(Vf)
026 |     inp = material.inputs(Vf)
027 |     rows = []
028 |
029 |     def add(model, values=None, status="ok", note=""):
030 |         row = dict.fromkeys(FIELDS, "")
031 |         row.update(case=material.id, unit=material.unit, Vf=Vf, Vm=Vm,
032 |                    model=model, status=status, note=note)
033 |         if values is not None:
034 |             row.update({k: v for k, v in values.items() if k in FIELDS})
035 |         rows.append(row)
036 |
037 |     mixtures = rule_of_mixtures(material.Ef, material.Em, Vf)
038 |     for name in ["voigt", "reuss"]:
039 |         add("simple_" + name, {"E": mixtures["E_" + name]},
040 |             status="idealization", note="1-D E mixture formula; separate from 3-D bounds.")
041 |     vr = isotropic_voigt_reuss(**inp)
042 |     for bound in ["lower", "upper"]:
043 |         add("isotropic_vr_" + bound, vr[bound], note="Isotropic stiffness/compliance bound.")
044 |     if Vf <= 0.1 + 1e-12:
045 |         # Snap away grid roundoff at the educational cutoff only.
046 |         add("eshelby_dilute", eshelby_dilute(**{**inp, "Vf": min(Vf, 0.1)}),
```

```
            status="low_fraction", note="No inclusion interactions; accuracy is contrast-dependent.")
    else:
        add("eshelby_dilute", status="not_evaluated",
            note="Outside the configured low-fraction display interval (Vf<=0.1).")
    add("mori_tanaka", mori_tanaka(**inp), note="Spherical inclusions in a connected isotropic matrix.")
    add("self_consistent", self_consistent(**inp), note="Both phases as spheres in the effective medium.")
    try:
        hs = hashin_shtrikman(**inp)
    except ValueError as exc:
        for bound in ["lower", "upper"]:
            add("hs_" + bound, status="not_applicable", note=str(exc))
    else:
        for bound in ["lower", "upper"]:
            add("hs_" + bound, hs[bound], note="Bound for an isotropic effective composite.")
    add("halpin_tsai_demo", halpin_tsai(material.Ef, material.Em, Vf, xi=material.xi),
        status="parameter_demo", note=f"xi={material.xi:g}; separate uncalibrated demonstration.")
    return rows


def fraction_grid(vf_max=0.6, points=61):
    vf_max = finite_scalar(vf_max, "vf_max")
    if not 0 < vf_max <= 1:
        raise ValueError("vf_max must be greater than 0 and at most 1.")
    if isinstance(points, bool) or not isinstance(points, int) or not 2 <= points <= 2001:
        raise ValueError("points must be an integer between 2 and 2001.")
    # Always include requested reporting fractions when inside the sweep.
    values = list(np.linspace(0, vf_max, points))
    values += [v for v in [0.05, 0.1, 0.3, 0.5, 0.6, 1.0] if v <= vf_max]
    return sorted(set(round(float(v), 12) for v in values))


def sweep(material, *, vf_max=0.6, points=61):
    return [row for vf in fraction_grid(vf_max, points) for row in evaluate(material, vf)]


def export_csv(rows, path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as stream:
        writer = csv.DictWriter(stream, fieldnames=FIELDS)
        writer.writeheader()
        writer.writerows(rows)
    return path


def key_rows(rows):
    return [r for r in rows if any(abs(r["Vf"] - v) < 1e-10 for v in [0, .05, .1, .3, .5, .6, 1])]
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Documents evaluation under scope rules and export of tidy result rows.
L2	Imports CSV writing support.
L3	Imports filesystem path handling.
L5	Imports NumPy for evenly spaced fraction grids.
L7-10	Imports the public model interfaces from the models package.
L11	Imports scalar and fraction checks for the orchestration layer.
L14-15	Defines the fixed CSV schema, including modulus values, missing-value status and solver diagnostics.
L18	Defines evaluation of one validated Material at one Vf.
L19-24	Documents the dilute display restriction, unavailable HS handling and propagation of other failures.
L25	Checks Vf and determines Vm=1-Vf.
L26	Builds the dictionary of named elastic-model inputs using the Material helper.
L27	Starts the list of result rows for this case/fraction.
L29	Defines a local row-building helper with default successful status.
L30	Initializes every table column to an empty string, making absent properties explicitly blank.
L31-32	Fills the case, unit, fractions, method name and interpretation fields for this row.
L33	Checks whether the caller supplied numerical properties.
L34	Copies only properties that appear in the table schema. For example, HT eta and xi are not separate CSV columns; xi remains in its note.
L35	Adds the completed row to the current result list.
L37	Computes the two simple scalar E mixtures once.
L38	Visits the equal-strain and equal-stress result names.
L39-40	Creates their labeled E-only rows with status idealization, preventing confusion with proper 3D bounds.
L41	Computes the proper isotropic stiffness/compliance bounds using the full phase inputs.
L42	Visits their lower and upper results.
L43	Adds all four elastic properties for each bounding tensor.
L44	Checks the educational dilute interval with a tiny floating-point allowance.
L45	Comment explaining the numerical handling at the display cutoff.
L46-47	Calls dilute only in this interval. A Vf infinitesimally above .1 from roundoff is clipped to .1 for the call; the row retains the original grid value.
L48	Handles fractions outside the low-fraction display interval.

Source line(s)	Explanation and connection to the project
L49-50	Adds a blank dilute row with its reason, rather than treating absence as zero or silently drawing extrapolation.
L51	Calculates and records the spherical Mori-Tanaka prediction.
L52	Calculates and records classical self-consistent properties and its solver diagnostics.
L53	Begins guarded evaluation of supported HS bounds.
L54	Requests the lower and upper bounds.
L55	Catches an unsupported-bound ValueError and stores it as exc.
L56	Creates an unavailable record for both bound labels.
L57	Records the explanation from the exception rather than a fabricated numeric bound.
L58	Runs only when the HS calculation succeeded.
L59	Visits both returned bounds.
L60	Adds their elastic properties and effective-isotropy note.
L61-62	Evaluates the explicitly supplied HT parameter and labels this E-only row as a separate parameter demonstration.
L63	Returns the ten labeled rows for this Vf.
L66	Defines generation of the fraction grid for a sweep.
L67	Checks that the maximum is a finite scalar.
L68	Requires a positive maximum no greater than one.
L69	Reports an invalid sweep extent.
L70	Requires an integer grid size from 2 to 2001 and rejects Boolean flags.
L71	Reports an unsuitable grid size before allocating or looping.
L72	Comment explaining why key reporting fractions are inserted even if not present on the equally spaced grid.
L73	Generates the requested equally spaced grid including zero and the maximum.
L74	Adds standard reporting fractions that fall within the requested sweep.
L75	Converts to floats, rounds at 12 decimal places, removes duplicates with a set, and sorts into increasing Vf order.
L78	Defines a sweep of one Material over the generated grid.
L79	Uses a nested comprehension to evaluate every fraction and flatten the ten rows per fraction into one list.
L82	Defines CSV export of any compatible row list.
L83	Normalizes the destination to a Path object.
L84	Creates missing parent directories, accepting an already-existing directory.
L85	Opens a UTF-8 output file; newline='' avoids extra CSV blank lines on Windows.

Source line(s)	Explanation and connection to the project
L86	Creates a dictionary-based CSV writer using the fixed field order.
L87	Writes the column-name row.
L88	Writes every result row without changing its values or missing-value markers.
L89	Returns the output path for callers that need to report where the file was saved.
L92	Defines selection of summary fractions from an existing sweep.
L93	Keeps rows within 1e-10 of the standard reporting fractions. This selects results; it does not recalculate or interpolate E.

analysis/plotting.py

Source file: analysis/plotting.py

What plotting does and does not do

This module plots values already computed by the models. Colors and line styles do not change a prediction. It saves figures without needing an open graphics window, which also supports automated report generation.

The first chart shows compatible isotropic bounds and effective-medium predictions. The second image has two panels: low-fraction inclusion models, and a separate scalar-mixture/HT demonstration. Keeping HT separate avoids claiming that its uncalibrated xi describes the same sphere geometry. Shaded regions are theoretical intervals, not experimental error bars or statistical confidence intervals. The plots do not rank model accuracy.

Numbered source

```
001 | """Export readable, scope-separated static figures without a display server."""
002 | from pathlib import Path
003 |
004 | import matplotlib
005 | matplotlib.use("Agg")
006 | import matplotlib.pyplot as plt
007 | import numpy as np
008 |
009 |
010 | COLORS = {"navy": "#153B57", "teal": "#087F8C", "gold": "#D28B22",
011 |           "purple": "#8561A8", "gray": "#718096"}
012 |
013 |
014 | def _series(rows, model, key="E"):
015 |     subset = [r for r in rows if r["model"] == model and r[key] != ""]
016 |     return np.array([r["Vf"] for r in subset]), np.array([r[key] for r in subset])
017 |
018 |
019 | def _style(ax, unit, title):
020 |     ax.set_title(title, loc="left", fontsize=12, fontweight="bold", pad=12)
021 |     ax.set_xlabel("Inclusion volume fraction, $V_f$")
022 |     ax.set_ylabel(f"Young's modulus, E ({unit})")
023 |     ax.grid(True, alpha=.18)
024 |     ax.spines[["top", "right"]].set_visible(False)
025 |     ax.set_axisbelow(True)
026 |
027 |
028 | def _save(fig, directory, name):
029 |     directory = Path(directory)
030 |     directory.mkdir(parents=True, exist_ok=True)
031 |     paths = []
032 |     for extension in ["png", "pdf"]:
033 |         target = directory / f"{name}.{extension}"
034 |         fig.savefig(target, dpi=180, bbox_inches="tight", facecolor="white")
035 |         paths.append(target)
036 |     plt.close(fig)
037 |     return paths
038 |
039 |
040 | def plot_case(material, rows, directory):
041 |     """Return paths for an isotropic comparison and separate demonstrations."""
042 |     plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
043 |                          "axes.labelcolor": COLORS["navy"], "text.color": COLORS["navy"]})
044 |     fig, ax = plt.subplots(figsize=(9, 4.8), layout="constrained")
045 |     x, low = _series(rows, "isotropic_vr_lower")
046 |     _, high = _series(rows, "isotropic_vr_upper")
047 |     ax.fill_between(x, low, high, color="#E7EDF2", label="Isotropic Voigt-Reuss interval")
048 |     hx, hl = _series(rows, "hs_lower")
049 |     _, hu = _series(rows, "hs_upper")
050 |     if len(hx):
```

```
        ax.fill_between(hx, hl, hu, color="#A8DADC", alpha=.65, label="Hashin-Shtrikman interval")
    x, y = _series(rows, "mori_tanaka")
    ax.plot(x, y, color=COLORS["teal"], lw=2.4, label="Mori-Tanaka")
    x, y = _series(rows, "self_consistent")
    ax.plot(x, y, color=COLORS["navy"], lw=2.2, ls="--", label="Self-consistent")
    _style(ax, material.unit, material.name)
    ax.legend(loc="best", frameon=False, fontsize=9)
    fig.supxlabel("Spherical, isotropic phases | Mathematical comparison; no experimental validation", fontsize=9)
    paths = _save(fig, directory, material.id + "_isotropic_comparison")

    fig, axes = plt.subplots(1, 2, figsize=(11, 4.4), layout="constrained")
    ax = axes[0]
    for model, label, color, ls in [
        ("eshelby_dilute", "Eshelby / dilute", COLORS["gold"], "-"),
        ("mori_tanaka", "Mori-Tanaka", COLORS["teal"], "--"),
        ("self_consistent", "Self-consistent", COLORS["navy"], ":"),
    ]:
        x, y = _series(rows, model)
        mask = x <= .1 + 1e-12
        ax.plot(x[mask], y[mask], label=label, color=color, ls=ls, lw=2)
    _style(ax, material.unit, "Low-fraction comparison")
    ax.legend(frameon=False, fontsize=9)
    ax = axes[1]
    for model, label, color, ls in [
        ("simple_voigt", "Simple equal-strain E", COLORS["gray"], "--"),
        ("simple_reuss", "Simple equal-stress E", COLORS["navy"], ":"),
        ("halpin_tsai_demo", f"Halpin-Tsai, xi={material.xi:g}", COLORS["purple"], "-"),
    ]:
        x, y = _series(rows, model)
        ax.plot(x, y, label=label, color=color, ls=ls, lw=2)
    _style(ax, material.unit, "Separate parameter demonstration")
    ax.legend(frameon=False, fontsize=9)
    fig.supxlabel("Dilute cutoff is illustrative | Halpin-Tsai xi is uncalibrated for the spherical cases", fontsize=9)
    paths += _save(fig, directory, material.id + "_model_demonstrations")
    return paths
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Documents static chart export and separation of different model roles.
L2	Imports portable output-path construction.
L4	Imports the plotting library before selecting its backend.
L5	Selects the Agg renderer, allowing file generation without a desktop graphics window.
L6	Imports the pyplot interface after backend selection.
L7	Imports arrays used for filtering and plotting data.
L10-11	Defines a consistent named color palette; these choices are visual only.
L14	Defines extraction of x/y arrays for a model, defaulting y to E.
L15	Keeps rows for that exact model with a nonblank requested property, so missing results are not converted to zeros.
L16	Returns matching NumPy arrays of Vf and the chosen property in their original row order.
L19	Defines common labels and presentation settings for an axis.
L20	Adds a left-aligned chart title with specified font weight and spacing.
L21	Labels the horizontal axis as dimensionless inclusion volume fraction.
L22	Labels E with the material's declared modulus unit.
L23	Adds a faint reference grid for reading values.
L24	Hides top/right borders to reduce visual clutter.
L25	Draws grids behind lines rather than over them.
L28	Defines saving one figure in both raster and vector formats.
L29	Converts the directory argument into a Path.
L30	Creates missing output directories.
L31	Starts the list of saved file paths.
L32	Loops over PNG for easy viewing and PDF for vector export.
L33	Builds a descriptive filename with the chosen extension.
L34	Saves with 180 dpi raster resolution, tight margins and a white background. Vector PDF lines retain vector quality.
L35	Records the saved file's path.
L36	Closes the figure to free resources during multiple-case runs.
L37	Returns the two saved paths.
L40	Defines generation of the pair of charts for one material case.
L41	Documents the two different comparison roles returned by this function.

Source line(s)	Explanation and connection to the project
L42-43	Sets common font and text/axis colors for consistent output.
L44	Creates the main isotropic comparison with layout management for labels.
L45	Extracts the lower isotropic VR bound and its fraction grid.
L46	Extracts the upper bound on the matching sweep grid; the underscore discards its duplicate x array.
L47	Shades the VR interval. This is a mechanical bound interval, not a measurement uncertainty band.
L48	Extracts available HS lower data and its own fraction array.
L49	Extracts HS upper data on the corresponding grid.
L50	Checks that any HS values exist before shading them.
L51	Shades the tighter supported HS interval, with transparency so overlapping regions remain visible.
L52	Extracts the Mori-Tanaka prediction.
L53	Plots it as a solid teal line with a descriptive legend entry.
L54	Extracts the self-consistent prediction.
L55	Plots it as a dashed navy line, distinguishable even when curves are close.
L56	Applies physical axis labels and the material title.
L57	Places a legend where it best fits without a surrounding box.
L58	Adds a figure-level assumption note to prevent the chart being read as experimental validation.
L59	Saves the main chart as PNG and PDF and starts the returned path list.
L61	Creates a second figure with two side-by-side panels.
L62	Selects the left panel.
L63-67	Defines the three low-fraction line styles for dilute, MT and SC; tuple unpacking assigns each style inside the loop.
L68	Extracts one model's available data.
L69	Builds a Boolean mask retaining only fractions through .1, with small numerical allowance.
L70	Draws the masked fraction/property arrays with the chosen style.
L71	Labels this explicitly as the low-fraction comparison.
L72	Adds the panel legend.
L73	Selects the right panel.
L74-78	Defines styles for the simple equal-strain, simple equal-stress and explicit-xi HT demonstrations.
L79	Extracts each scalar model series.
L80	Draws the scalar demonstration across the available sweep.
L81	Labels the panel as a separate parameter demonstration, not equivalent spherical homogenization.
L82	Adds its legend.

Source line(s)	Explanation and connection to the project
L83	States the dilute and xi limitations at figure level.
L84	Saves the second chart in both formats and extends the collected paths.
L85	Returns all four output paths for the two charts.

main.py

Source file: `main.py`

Purpose and execution flow

This is the application's entry point. It reads your choices, selects material cases, delegates calculations, and saves or prints results. It contains no new modulus law. `python main.py` defaults to one case at Vf=.05; `python main.py --all --report` sweeps all cases and creates the report.

`parser()` defines accepted terminal options; `run()` executes the chosen workflow. A successful return code is 0; a handled error exits with code 2. ROOT is based on this file's location, so built-in data/output defaults work even when the program is launched from another directory. User-supplied relative paths still follow the terminal's current working directory.

The `--vf`/`--vm` sweep check compares Vf's numeric value to its default .05; it does not remember whether the user explicitly typed `--vf 0.05`. That exact value is accepted in sweep mode but has no effect on the sweep. Use `--vf-max` to change the sweep extent. The xi-override comment mentions schema checks, but the code actually validates the overridden parameter by calling HT at Vf=0; it does not call Material.from_dict again. These details matter when explaining the real implementation rather than an idealized description.

Numbered source

```
001 | """Run a single case, export comparisons, or generate the project report."""
002 | import argparse
003 | from dataclasses import replace
004 | import json
005 | from pathlib import Path
006 | import sys
007 |
008 | from analysis.compare_models import evaluate, export_csv, key_rows, sweep
009 | from utilities.input_checks import fractions
010 | from utilities.materials import load_materials
011 |
012 | ROOT = Path(__file__).resolve().parent
013 |
014 |
015 | def parser():
016 |     p = argparse.ArgumentParser(description="Analytical modulus prediction for idealized isotropic
        composites.")
017 |     p.add_argument("--data", type=Path, default=ROOT / "data/materials.json", help="Material JSON
        file.")
018 |     p.add_argument("--case", default="stiff_inclusions", help="Case id from the material file.")
019 |     p.add_argument("--list-cases", action="store_true", help="Show available inputs and their
        provenance.")
020 |     p.add_argument("--vf", type=float, default=.05, help="Single-run inclusion volume fraction,
        default 0.05.")
021 |     p.add_argument("--vm", type=float, help="Optional single-run matrix fraction; must equal 1-vf.")
022 |     p.add_argument("--xi", type=float, help="Override the Halpin-Tsai demonstration parameter.")
023 |     p.add_argument("--compare", action="store_true", help="Sweep the selected material case.")
024 |     p.add_argument("--all", action="store_true", help="Sweep every material case.")
025 |     p.add_argument("--vf-max", type=float, default=.6, help="Sweep maximum, default 0.6; must be in
        (0,1].")
026 |     p.add_argument("--points", type=int, default=61, help="Sweep grid size, default 61; key
        fractions also included.")
027 |     p.add_argument("--report", action="store_true", help="Generate comparisons plus Markdown and PDF
         report.")
028 |     p.add_argument("--output", type=Path, default=ROOT / "outputs", help="Output directory.")
029 |     return p
030 |
031 |
032 | def run(argv=None):
033 |     p = parser()
034 |     args = p.parse_args(argv)
035 |     try:
```

```
        cases = load_materials(args.data)
        if args.list_cases:
            for m in cases.values():
                print(f"{m.id}: {m.name}\n  Ef={m.Ef:g}, Em={m.Em:g} {m.unit}; "
                      f"nu_f={m.nu_f:g}, nu_m={m.nu_m:g}\n  {m.source_status}: {m.source_note}")
            return 0
        if not args.all and args.case not in cases:
            raise ValueError(f"Unknown case '{args.case}'. Choose: {', '.join(cases)}")
        selected = list(cases.values()) if args.all else [cases[args.case]]
        if args.xi is not None:
            # Revalidate the override through the same schema checks.
            selected = [replace(m, xi=args.xi) for m in selected]
            from models import halpin_tsai
            for m in selected:
                halpin_tsai(m.Ef, m.Em, 0, xi=m.xi)
        out = args.output.resolve()
        out.mkdir(parents=True, exist_ok=True)
        if args.compare or args.all or args.report:
            if args.vm is not None or args.vf != .05:
                raise ValueError("--vf/--vm apply to single runs; use --vf-max for a comparison sweep.")
            from analysis.plotting import plot_case
            all_rows, generated = [], []
            for material in selected:
                rows = sweep(material, vf_max=args.vf_max, points=args.points)
                all_rows.extend(rows)
                export_csv(rows, out / "tables" / f"{material.id}_all_results.csv")
                export_csv(key_rows(rows), out / "tables" / f"{material.id}_key_fractions.csv")
                generated += [str(p.relative_to(out)) for p in plot_case(material, rows, out / "figures")]
                print(f"Completed {material.id}: {len(rows)//10} fractions, tables and figures.")
            export_csv(all_rows, out / "tables/all_cases_results.csv")
            manifest = {
                "project": "Analytical composite modulus", "geometry": "spherical",
                "data_file": str(args.data.resolve()), "materials": [m.to_dict() for m in selected],
                "vf_max": args.vf_max, "requested_grid_points": args.points,
                "dilute_display_cutoff": .1, "experimental_validation": False,
                "halpin_tsai_role": "separate uncalibrated parameter demonstration",
                "figures": generated,
            }
            (out / "run_manifest.json").write_text(json.dumps(manifest, indent=2), encoding="utf-8")
            if args.report:
                from report.build_report import build_report
                report_files = build_report(selected, all_rows, out, ROOT)
                for path in report_files:
                    print(f"Report: {path}")
            print(f"Results saved to: {out}")
        else:
            Vf, _ = fractions(args.vf, args.vm)
            material = selected[0]
            rows = evaluate(material, Vf)
            print(f"\n{material.name}\nInputs: Ef={material.Ef:g}, Em={material.Em:g} {material.unit}, "
                  f"nu_f={material.nu_f:g}, nu_m={material.nu_m:g}, Vf={Vf:g}, Vm={1-Vf:g}")
            print(f"Data status: {material.source_status}\n{material.source_note}\n")
            print(f"{'Method':<24} {'E ('+material.unit+')':>13}  Status")
            for row in rows:
                value = f"{row['E']:.6f}" if row["E"] != "" else "--"
                print(f"{row['model']:<24} {value:>13}  {row['status']}")
            export_csv(rows, out / "tables" / f"{material.id}_single.csv")
            payload = {"material": material.to_dict(), "Vf": Vf, "results": rows}
            (out / f"{material.id}_single.json").write_text(json.dumps(payload, indent=2), encoding="utf-8")
            print("\nDilute appears only for Vf<=0.1. HT is a separate parameter demonstration.")
        return 0
    except (ValueError, OSError, RuntimeError, ImportError) as exc:
        p.exit(2, f"Error: {exc}\n")


if __name__ == "__main__":
    sys.exit(run())
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Documents the three workflow modes: single case, comparisons and report.
L2	Imports command-line argument parsing.
L3	Imports creation of a modified dataclass copy for the xi override.
L4	Imports JSON serialization for manifests and single-run outputs.
L5	Imports path handling.
L6	Imports process exit support.
L8	Imports evaluation, CSV writing, key-fraction selection and sweep orchestration.
L9	Imports fraction validation used by single runs.
L10	Imports the material JSON loader.
L12	Finds the absolute project folder from main.py itself, giving portable built-in paths.
L15	Defines the terminal-option parser.
L16	Creates the parser with a short description of the supported physical scope.
L17	Adds --data, converting its argument to Path and defaulting to the included material database.
L18	Adds --case, selecting stiff_inclusions by default.
L19	Adds the flag that prints available cases and their sources without running a calculation.
L20	Adds a floating-point single-run Vf, default 0.05.
L21	Adds optional Vm; if absent the model preparation uses 1-Vf.
L22	Adds an optional explicit HT-parameter override.
L23	Adds the flag requesting a fraction sweep for the selected case.
L24	Adds the flag selecting every material case and activating sweep mode.
L25	Adds the sweep endpoint Vf_max, default 0.6.
L26	Adds the requested grid size, default 61; the grid helper may insert additional key fractions.
L27	Adds the report flag, which also activates comparison mode.
L28	Adds the output-folder option with a project-relative built-in default.
L29	Returns the configured parser object; this alone does not parse or run the models.
L32	Defines the runnable workflow with optional arguments supplied programmatically by a caller.
L33	Builds the parser.
L34	Reads terminal arguments, or an explicitly supplied argv list, into the args object.
L35	Begins handling of expected data, filesystem, solver and dependency failures.
L36	Loads and validates all material cases from the chosen file.

Source line(s)	Explanation and connection to the project
L37	Checks for the list-only mode.
L38	Visits each material record.
L39-40	Prints identifier, name, E values, Poisson ratios and provenance; creates readable line breaks and :g uses compact numeric formatting.
L41	Returns a successful status after listing, without calculations.
L42	When not selecting all cases, checks that the requested identifier exists.
L43	Reports an unknown identifier and lists valid alternatives.
L44	Creates either the complete case list or a one-element list for a selected case.
L45	Checks whether xi was overridden explicitly.
L46	Comment describing the intent to revalidate the override; the actual following check is an HT call rather than the JSON factory.
L47	Creates new frozen-dataclass copies with the new xi, leaving original records unchanged.
L48	Imports the HT validator/calculation only in this override branch.
L49	Checks the override for every selected case.
L50	Calls HT at Vf=0 to validate xi without needing a mixed-phase prediction; it does not calibrate xi.
L51	Resolves the output location to an absolute path.
L52	Creates the output directory if needed.
L53	Enters sweep mode when any of compare, all or report is active.
L54	Checks for incompatible single-run fraction choices; a Vf numerically equal to the .05 default is not rejected.
L55	Explains that sweep extent must be set with --vf-max rather than a single Vf/Vm pair.
L56	Imports plotting only when needed, so basic calculations do not require loading Matplotlib.
L57	Initializes the combined result rows and list of generated plot paths.
L58	Loops over the chosen materials.
L59	Evaluates the complete grid for one material.
L60	Extends the combined table with that material's rows.
L61	Writes all per-fraction results to a case-specific CSV file.
L62	Writes a smaller CSV containing key reporting fractions from the same computed rows.
L63	Generates both charts and records paths relative to the output directory in the manifest.
L64	Prints completion with rows//10 as the fraction count, relying on evaluate returning ten rows per fraction.
L65	Writes the combined all-selected-cases table.
L66	Begins the reproducibility metadata dictionary.

Source line(s)	Explanation and connection to the project
L67	Records the project label and supported spherical geometry.
L68	Records the exact source data path and the selected material records, including any xi override.
L69	Records the requested sweep endpoint and base grid size, which can differ from the final grid after key fractions are inserted.
L70	Records the educational dilute cutoff and explicitly states that there is no experimental validation.
L71	Records the distinct uncalibrated role of HT.
L72	Stores the generated figure paths.
L73	Closes the manifest dictionary.
L74	Serializes it as indented UTF-8 JSON to run_manifest.json.
L75	Checks whether report generation was requested.
L76	Imports the report builder only for that optional workflow.
L77	Passes actual rows and inputs to the builder; that module runs the verification suite before producing the report.
L78	Visits the returned report paths.
L79	Prints each generated report location.
L80	Prints the common results directory after the selected sweeps finish.
L81	Enters single-run mode when no sweep/report flag is active.
L82	Validates the single-run Vf and optional supplied Vm; the returned Vm is not otherwise needed here.
L83	Takes the one selected material.
L84	Calculates its ten labeled result rows at the requested Vf.
L85-86	Prints the material name and actual numerical inputs, including the complementary Vm.
L87	Prints provenance so illustrative inputs cannot be silently read as measured data.
L88	Prints aligned table column labels. <24 means left-align in 24 characters and >13 means right-align in 13.
L89	Visits the ten result rows.
L90	Formats available E values to six decimal places and unavailable values as -- rather than zero.
L91	Prints the model, formatted E and status without altering the stored numeric precision.
L92	Writes the single-run CSV table.
L93	Builds a JSON object combining the material inputs, Vf and all result rows.
L94	Writes that object to a case-specific JSON file.
L95	Prints the dilute and HT interpretation reminder.
L96	Returns success when the chosen workflow completes normally.
L97	Catches expected validation, disk, solver and missing-dependency failures.

Source line(s)	Explanation and connection to the project
L98	Prints a concise error through argparse and exits with status 2 instead of silently continuing.
L101	Checks whether main.py was executed directly rather than imported as a module.
L102	Runs the workflow and passes its return value to the operating system as the process exit status.

tests/test_limits.py

Source file: `tests/test_limits.py`

Theory behind the tests

A limit test uses a case whose answer is already known without trusting the general prediction: no inclusions must give the matrix; identical phases must give their common material; a reversible elastic conversion should return its starting values. These checks detect mistakes but do not establish experimental accuracy. `unittest` finds methods beginning with `test_`.

`assertAlmostEqual(..., places=12)` checks decimal-place agreement of the supplied values. Relative ratios are used for some checks so different modulus scales can be tested. `assertRaises` expects failure; a model accepting an invalid value makes that test fail. `subTest` identifies which input case failed without turning every input into a separate test method.

Numbered source

```
001 | import unittest
002 | import warnings
003 |
004 | import numpy as np
005 |
006 | from models import (
007 |     eshelby_dilute, halpin_tsai, hashin_shtrikman, mori_tanaka,
008 |     reuss, self_consistent, sphere_eshelby, voigt,
009 | )
010 | from utilities.elastic_properties import from_bulk_shear, to_bulk_shear
011 |
012 |
013 | class LimitTests(unittest.TestCase):
014 |     def test_conversion_round_trip(self):
015 |         for E in [1, 70, 200e9]:
016 |             for nu in [-.3, 0, .35, .49]:
017 |                 with self.subTest(E=E, nu=nu):
018 |                     result = from_bulk_shear(*to_bulk_shear(E, nu))
019 |                     self.assertAlmostEqual(result["E"] / E, 1, places=12)
020 |                     self.assertAlmostEqual(result["nu"], nu, places=12)
021 |
022 |     def test_phase_endpoints(self):
023 |         for model in [mori_tanaka, self_consistent]:
024 |             for Vf, expected in [(0, 3.5), (1, 70)]:
025 |                 with self.subTest(model=model.__name__, Vf=Vf):
026 |                     result = model(70, 3.5, Vf, nu_f=.2, nu_m=.35)
027 |                     self.assertAlmostEqual(result["E"], expected, places=11)
028 |         for Vf, expected in [(0, 3.5), (1, 70)]:
029 |             for fn in [voigt, reuss]:
030 |                 self.assertAlmostEqual(fn(70, 3.5, Vf), expected)
031 |             self.assertAlmostEqual(halpin_tsai(70, 3.5, Vf, xi=2)["E"], expected)
032 |             for bound in hashin_shtrikman(70, 3.5, Vf, nu_f=.2, nu_m=.35).values():
033 |                 self.assertAlmostEqual(bound["E"], expected)
034 |
035 |     def test_identical_phases(self):
036 |         for fn in [mori_tanaka, self_consistent, eshelby_dilute]:
037 |             self.assertAlmostEqual(fn(7, 7, .05, nu_f=.32, nu_m=.32)["E"], 7)
038 |
039 |     def test_invalid_fractions_and_moduli(self):
040 |         for value in [-.1, 1.1, float("nan"), float("inf"), True, [.1]]:
041 |             with self.subTest(value=value), self.assertRaises(ValueError):
042 |                 voigt(70, 3.5, value)
043 |         for E in [0, -1, float("nan"), float("inf")]:
044 |             with self.assertRaises(ValueError):
045 |                 voigt(E, 3.5, .2)
046 |         with self.assertRaises(ValueError):
047 |             voigt(70, 3.5, .2, Vm=.7)
048 |
049 |     def test_invalid_poisson_ratios(self):
050 |         for nu in [-1, .5, float("nan")]:
```

```
            with self.assertRaises(ValueError):
                mori_tanaka(70, 3.5, .2, nu_f=.2, nu_m=nu)

    def test_dilute_warning_and_rejected_endpoint(self):
        with warnings.catch_warnings(record=True) as caught:
            warnings.simplefilter("always")
            eshelby_dilute(70, 3.5, .3, nu_f=.2, nu_m=.35)
        self.assertEqual(len(caught), 1)
        with self.assertRaises(ValueError):
            eshelby_dilute(70, 3.5, 1, nu_f=.2, nu_m=.35)

    def test_halpin_tsai_reuss_limit(self):
        for vf in [0, .1, .5, 1]:
            self.assertAlmostEqual(halpin_tsai(70, 3.5, vf, xi=0)["E"], reuss(70, 3.5, vf))
        with self.assertRaises(ValueError):
            halpin_tsai(70, 3.5, .2, xi=-1)

    def test_solver_failure_is_reported(self):
        with self.assertRaises(RuntimeError):
            self_consistent(70, 3.5, .4, nu_f=.2, nu_m=.35, max_iterations=1)
        with self.assertRaises(ValueError):
            self_consistent(70, 3.5, .4, nu_f=.2, nu_m=.35, tolerance=0)

    def test_equal_shear_phase_case(self):
        result = self_consistent(12, 13, .4, nu_f=.2, nu_m=.3)
        self.assertAlmostEqual(result["G"], 5, places=12)

    def test_kelvin_and_cartesian_eshelby_agree(self):
        basis = np.zeros((6, 3, 3))
        for i in range(3):
            basis[i, i, i] = 1
        for a, (i, j) in enumerate([(1, 2), (0, 2), (0, 1)], 3):
            basis[a, i, j] = basis[a, j, i] = 1 / np.sqrt(2)
        for nu in [-.3, 0, .2, .35, .49]:
            S4 = sphere_eshelby(nu, notation="cartesian")
            converted = np.einsum("aij,ijkl,bkl->ab", basis, S4, basis)
            np.testing.assert_allclose(converted, sphere_eshelby(nu), atol=1e-14)
        with self.assertRaises(ValueError):
            sphere_eshelby(.3, notation="voigt")


if __name__ == "__main__":
    unittest.main()
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Imports Python's built-in unit-testing framework.
L2	Imports warning capture, needed to verify dilute extrapolation warnings.
L4	Imports array construction and numerical comparison helpers.
L6-9	Imports the public prediction, bounds and tensor functions under test.
L10	Imports the forward and inverse elastic conversions.
L13	Defines a collection of tests with unittest assertion methods inherited from TestCase.
L14	Defines a test that converts E,nu to K,G and back.
L15	Visits different modulus scales, including a value expressed at a Pa-like magnitude.
L16	Visits negative, zero, ordinary positive and near-incompressible Poisson ratios inside the supported interval.
L17	Labels each combination so failures identify their inputs.
L18	Calls the forward conversion, unpacks its pair into the inverse conversion, and stores the result.
L19	Checks recovered E divided by original E equals one to 12 decimal places, making the comparison relative to scale.
L20	Checks recovery of the original dimensionless nu.
L22	Defines endpoint checks for pure-phase limits.
L23	Visits the MT and SC model functions, passing functions themselves as values.
L24	Pairs Vf=0 with matrix E and Vf=1 with inclusion E.
L25	Labels the current model and endpoint as a subtest.
L26	Runs the model with explicit constituent Poisson ratios.
L27	Checks the returned E against the exact pure-phase value.
L28	Repeats the endpoint pairs for additional model interfaces.
L29	Visits both simple mixture functions.
L30	Checks their scalar E endpoints.
L31	Checks HT endpoints for the explicit xi=2 demonstration.
L32	Visits the lower and upper HS dictionaries at each pure-phase endpoint.
L33	Checks that both bound values collapse to the present phase E.
L35	Defines the equal-constituent-property check.
L36	Visits MT, SC and dilute at a supported low Vf.
L37	Checks that combining identical isotropic phases cannot create a different E.
L39	Defines rejection tests for invalid fractions and moduli.

Source line(s)	Explanation and connection to the project
L40	Lists out-of-range, nonfinite, Boolean and array fraction inputs that must be rejected.
L41	Labels each bad value and expects a ValueError.
L42	Attempts the mixture calculation, which must trigger the expected validation error.
L43	Visits zero, negative and nonfinite constituent Young's moduli.
L44	Expects a validation error for each.
L45	Attempts a calculation using that invalid inclusion modulus.
L46	Expects contradictory fractions to fail.
L47	Supplies Vf=.2 and Vm=.7, whose sum is .9 rather than one.
L49	Defines rejection tests at unsupported Poisson ratios.
L50	Visits the stability endpoints and NaN.
L51	Expects each to raise ValueError.
L52	Attempts MT with an invalid matrix nu, verifying that shared phase preparation blocks it.
L54	Defines checks of dilute warnings and the rejected pure-inclusion endpoint.
L55	Captures warnings in a local list rather than printing them during the test.
L56	Ensures the warning is captured even if the same warning was previously emitted.
L57	Calls dilute above the teaching cutoff, where a warning should be emitted.
L58	Checks that exactly one warning was captured.
L59	Expects Vf=1 to raise an error rather than return a patched dilute value.
L60	Makes that pure-inclusion dilute call.
L62	Defines the HT-to-Reuss mathematical limiting check.
L63	Visits several fractions including both endpoints.
L64	Checks that HT with xi=0 equals the independent simple harmonic-mixture function.
L65	Expects a negative xi to be rejected.
L66	Attempts the unsupported negative-parameter call.
L68	Defines checks that solver failures and invalid settings are reported.
L69	Expects a RuntimeError when an unrealistically small iteration budget cannot converge.
L70	Calls SC with only one iteration for a nontrivial material case.
L71	Expects zero tolerance to be rejected as an invalid solver setting.
L72	Makes the zero-tolerance call.
L74	Defines a special case with equal phase shear moduli but different bulk moduli.
L75	Chooses E,nu pairs that both give G=5, checking that the shear bracket may have zero width without breaking the solver.

Source line(s)	Explanation and connection to the project
L76	Checks the common G is recovered.
L78	Defines an independent check of Kelvin/Cartesian Eshelby representations.
L79	Allocates six 3x3 basis tensors, initially zero.
L80	Loops over the three normal basis directions.
L81	Sets each normal basis tensor's matching diagonal component to one.
L82	Visits the shear-pair ordering 23,13,12, assigning Kelvin indices 3,4,5.
L83	Places symmetric 1/sqrt(2) entries in both off-diagonal positions, creating orthonormal shear basis tensors.
L84	Checks several host Poisson ratios.
L85	Obtains the full Cartesian fourth-order Eshelby tensor.
L86	Contracts the Cartesian tensor with basis tensors on both sides to build an independent Kelvin representation.
L87	Checks agreement with the directly assembled Kelvin S, detecting missing factors of two in shear terms.
L88	Expects an unsupported representation string to fail.
L89	Requests engineering-style voigt notation, which this implementation deliberately does not provide.
L92	Checks whether this test file was executed directly.
L93	Runs the unittest test runner for the file.

tests/test_benchmarks.py

Source file: `tests/test_benchmarks.py`

What this file actually validates

The comment in your originally proposed tree said 'published reference results'. The delivered file does **not** contain measured or published numerical benchmark datasets. It has an independently calculated rational fixture and 120 seeded material-property cases checking equations, invariances and applicable bounds. Call these analytical implementation checks, not experimental validation.

The exact fixture has `Kf=100`, `Gf=40`, `Km=10`, `Gm=5`, `Vf=1/4`. E and nu are derived from those inputs. Rational arithmetic stores fractions exactly, so it provides a useful reference separate from floating-point tensor assembly. Expected MT values are `K=1460/101` and `G=2635/352` in the chosen modulus unit.

The random seed makes the study repeatable; random does not mean measured. The tests evaluate scalar reductions separately from tensor assembly, but both implement the same underlying analytical model. Agreement checks coding consistency, not whether the model matches a real microstructure.

Numbered source

```
001 | """Analytical fixtures and seeded checks; no experimental accuracy claim."""
002 | from fractions import Fraction as F
003 | import unittest
004 |
005 | import numpy as np
006 |
007 | from models import (
008 |     eshelby_dilute, hashin_shtrikman, isotropic_voigt_reuss,
009 |     mori_tanaka, self_consistent,
010 | )
011 | from utilities.elastic_properties import to_bulk_shear
012 |
013 |
014 | class BenchmarkTests(unittest.TestCase):
015 |     def test_exact_rational_fixture(self):
016 |         result = mori_tanaka(1800 / 17, 90 / 7, .25, nu_f=11 / 34, nu_m=2 / 7)
017 |         expected_K = F(10) + F(1, 4) * 90 / (1 + F(3, 4) * 90 / F(50, 3))
018 |         expected_G = F(5) + F(1, 4) * 35 / (1 + F(3, 4) * 35 / F(125, 12))
019 |         self.assertAlmostEqual(result["K"], float(expected_K), places=11)
020 |         self.assertAlmostEqual(result["G"], float(expected_G), places=11)
021 |
022 |     def test_120_seeded_material_cases(self):
023 |         rng = np.random.default_rng(42)
024 |         for case in range(120):
025 |             Ef, Em = 10 ** rng.uniform(-2, 4, 2)
026 |             nuf, num = rng.uniform(-.3, .48, 2)
027 |             vf = float(rng.uniform(.001, .999))
028 |             inp = dict(Ef=Ef, Em=Em, Vf=vf, nu_f=nuf, nu_m=num)
029 |             with self.subTest(case=case):
030 |                 kf, gf = to_bulk_shear(Ef, nuf)
031 |                 km, gm = to_bulk_shear(Em, num)
032 |                 z = gm * (9 * km + 8 * gm) / (6 * (km + 2 * gm))
033 |                 mt = mori_tanaka(**inp)
034 |                 expected = [
035 |                     km + vf * (kf - km) / (1 + (1 - vf) * (kf - km) / (km + 4 * gm / 3)),
036 |                     gm + vf * (gf - gm) / (1 + (1 - vf) * (gf - gm) / (gm + z)),
037 |                 ]
038 |                 np.testing.assert_allclose([mt["K"], mt["G"]], expected, rtol=2e-10)
039 |                 dilute = eshelby_dilute(**{**inp, "Vf": .02})
040 |                 np.testing.assert_allclose([dilute["K"], dilute["G"]], [
041 |                     km + .02 * (kf - km) * (km + 4 * gm / 3) / (kf + 4 * gm / 3),
042 |                     gm + .02 * (gf - gm) * (gm + z) / (gf + z),
043 |                 ], rtol=2e-10)
044 |                 sc = self_consistent(**inp, tolerance=1e-11)
045 |                 k, g = sc["K"], sc["G"]
046 |                 zg = g * (9 * k + 8 * g) / (6 * (k + 2 * g))
```

```
                residuals = [
                    vf * (kf - k) / (kf + 4 * g / 3) + (1 - vf) * (km - k) / (km + 4 * g / 3),
                    vf * (gf - g) / (gf + zg) + (1 - vf) * (gm - g) / (gm + zg),
                ]
                np.testing.assert_allclose(residuals, 0, atol=1.1e-11)
                swapped = self_consistent(Em, Ef, 1-vf, nu_f=num, nu_m=nuf, tolerance=1e-11)
                self.assertAlmostEqual(swapped["E"] / sc["E"], 1, places=8)
                scaled = self_consistent(**{**inp, "Ef": Ef*1e9, "Em": Em*1e9}, tolerance=1e-11)
                self.assertAlmostEqual(scaled["E"] / (sc["E"]*1e9), 1, places=8)
                vr = isotropic_voigt_reuss(**inp)
                if (kf-km)*(gf-gm) >= 0:
                    hs = hashin_shtrikman(**inp)
                    bound = "lower" if km <= kf and gm <= gf else "upper"
                    for key in ["E", "K", "G"]:
                        self.assertLessEqual(vr["lower"][key]*(1-1e-8), hs["lower"][key])
                        self.assertLessEqual(hs["lower"][key]*(1-1e-8), sc[key])
                        self.assertLessEqual(sc[key], hs["upper"][key]*(1+1e-8))
                        self.assertLessEqual(hs["upper"][key], vr["upper"][key]*(1+1e-8))
                        np.testing.assert_allclose(mt[key], hs[bound][key], rtol=1e-8)
                else:
                    with self.assertRaises(ValueError):
                        hashin_shtrikman(**inp)


if __name__ == "__main__":
    unittest.main()
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Documents that these fixtures and generated cases make no experimental accuracy claim.
L2	Imports exact rational-number arithmetic under the short alias F.
L3	Imports the test framework.
L5	Imports repeatable random generation, arrays and numerical comparisons.
L7-10	Imports the model functions and isotropic comparison bounds.
L11	Imports phase conversion for independently written scalar equations.
L14	Defines the benchmark-check test collection.
L15	Defines the exact rational fixture test.
L16	Calls the tensor MT implementation using E,nu values corresponding to the known K,G fixture.
L17	Computes expected bulk MT using exact fractions and the independently written scalar equation.
L18	Computes expected shear MT likewise; 125/12 equals Gm+zeta_m for this fixture.
L19	Compares calculated bulk modulus with the exact reference converted to float.
L20	Compares the shear modulus with its independent rational reference.
L22	Defines the repeatable 120-case implementation study.
L23	Initializes the random-number generator with seed 42 so the same cases recur on each run.
L24	Repeats the checks for 120 generated material/fraction combinations.
L25	Generates positive Ef and Em logarithmically between 1e-2 and 1e4, spanning substantial phase contrast.
L26	Generates phase Poisson ratios within the supported stable interval.
L27	Generates an interior fraction to avoid testing only trivial endpoints.
L28	Assembles named arguments for the public model calls.
L29	Labels the generated case number in test failures.
L30	Converts inclusion E,nu for the scalar reference equations.
L31	Converts matrix E,nu for the same purpose.
L32	Independently evaluates the matrix's spherical shear parameter directly from its formula.
L33	Runs the tensor-based MT implementation.
L34	Starts the independently written scalar expected-value pair.
L35	Evaluates scalar spherical MT bulk modulus.
L36	Evaluates scalar spherical MT shear modulus.
L37	Completes that expected-value pair.

Source line(s)	Explanation and connection to the project
L38	Checks the tensor MT bulk/shear pair against the independent scalar pair at relative tolerance 2e-10.
L39	Runs dilute at Vf=.02, replacing only the generated fraction to stay in a low-fraction case.
L40	Starts a comparison between tensor dilute and an independently written scalar pair.
L41	Evaluates the scalar spherical dilute bulk formula at the same .02 fraction.
L42	Evaluates its shear counterpart.
L43	Completes the comparison with the same tight relative tolerance.
L44	Runs SC with an explicitly tightened 1e-11 residual tolerance.
L45	Extracts the converged K,G.
L46	Independently recomputes the effective host's shear parameter.
L47	Starts the direct governing-equation residual list.
L48	Evaluates the two-phase bulk residual at the reported solution.
L49	Evaluates the two-phase shear residual.
L50	Completes the residual pair.
L51	Checks that both equations are satisfied within absolute tolerance 1.1e-11.
L52	Swaps the phase labels and complements Vf for the symmetric classical SC scheme.
L53	Checks that this relabeling does not change effective E. This symmetry is not imposed on matrix-host MT.
L54	Multiplies both phase E values by 1e9 and resolves SC, simulating a change from GPa to Pa.
L55	Checks that effective E scales by exactly the same factor within numerical tolerance.
L56	Computes proper isotropic VR bounds for a consistency reference.
L57	Checks whether phase contrasts in K and G have the same sign, making the supported HS ordering applicable.
L58	Computes HS bounds only for such well-ordered phases.
L59	Selects the HS boundary expected to coincide with MT based on whether the matrix is soft or stiff.
L60	Checks E, K and G individually; nu is deliberately not treated as separately bounded.
L61	Checks that the HS lower bound is not below the Reuss lower bound, allowing small numerical tolerance.
L62	Checks SC does not fall below the applicable HS lower value for these cases.
L63	Checks SC does not exceed the HS upper value.
L64	Checks the HS upper value does not exceed the Voigt stiffness upper bound.
L65	Checks the MT value coincides with its expected HS boundary.
L66	Handles generated cross-ordered phase pairs.

Source line(s)	Explanation and connection to the project
L67	Expects the supported HS implementation to reject such a pair rather than claim a wrong bound.
L68	Makes the bound call that must raise ValueError.
L71	Checks direct execution of the test file.
L72	Runs its tests through unittest.

tests/test_project.py

Source file: `tests/test_project.py`

Why the complete project includes this extra test file

Correct equations are not sufficient if inputs are silently mislabeled, missing values become zeros, files overwrite one another, or launching from another directory breaks default data paths. These integration tests check those behaviors. Temporary directories prevent these checks from creating permanent test results in the user project. `subprocess` launches a fresh Python process, testing the real command-line entry point.

Numbered source

```
001 | import csv
002 | import json
003 | from pathlib import Path
004 | import subprocess
005 | import sys
006 | import tempfile
007 | import unittest
008 |
009 | from analysis.compare_models import evaluate, export_csv, fraction_grid, sweep
010 | from utilities.materials import load_materials, Material
011 |
012 | ROOT = Path(__file__).resolve().parents[1]
013 |
014 |
015 | class ProjectTests(unittest.TestCase):
016 |     @classmethod
017 |     def setUpClass(cls):
018 |         cls.material = load_materials(ROOT / "data/materials.json")["stiff_inclusions"]
019 |
020 |     def test_data_provenance_is_present(self):
021 |         for m in load_materials(ROOT / "data/materials.json").values():
022 |             self.assertEqual(m.source_status, "illustrative")
023 |             self.assertTrue(m.source_note)
024 |
025 |     def test_unsupported_geometry_and_unknown_keys_rejected(self):
026 |         raw = self.material.to_dict()
027 |         raw.pop("unit")
028 |         for change in [{"geometry": "cylinder"}, {"Ef_typo": 70}, {"Ef": -1}]:
029 |             with self.assertRaises(ValueError):
030 |                 Material.from_dict({**raw, **change})
031 |
032 |     def test_grid_includes_requested_key_fractions(self):
033 |         values = fraction_grid(.6, 8)
034 |         for vf in [0, .05, .1, .3, .5, .6]:
035 |             self.assertIn(vf, values)
036 |         self.assertEqual(values, sorted(set(values)))
037 |
038 |     def test_bad_grids_rejected(self):
039 |         for maximum, points in [(0, 61), (1.1, 61), (.6, 1), (.6, 3000)]:
040 |             with self.assertRaises(ValueError):
041 |                 fraction_grid(maximum, points)
042 |
043 |     def test_dilute_is_missing_above_cutoff(self):
044 |         rows = evaluate(self.material, .3)
045 |         dilute = next(r for r in rows if r["model"] == "eshelby_dilute")
046 |         self.assertEqual(dilute["E"], "")
047 |         self.assertEqual(dilute["status"], "not_evaluated")
048 |         self.assertEqual(len(rows), 10)
049 |
050 |     def test_cross_ordered_bounds_are_explicitly_unavailable(self):
051 |         raw = self.material.to_dict()
052 |         raw.pop("unit")
053 |         # Larger E but lower K: phase ordering differs between K and G.
054 |         m = Material.from_dict({**raw, "Ef": 12, "Em": 10, "nu_f": 0, "nu_m": .45})
055 |         rows = evaluate(m, .05)
```

```
        hs = [r for r in rows if r["model"].startswith("hs_")]
        self.assertEqual(len(hs), 2)
        self.assertTrue(all(r["status"] == "not_applicable" and r["E"] == "" for r in hs))

    def test_csv_keeps_missing_values_and_notes(self):
        with tempfile.TemporaryDirectory() as d:
            target = export_csv(sweep(self.material, vf_max=.2, points=3), Path(d)/"data.csv")
            with target.open() as f:
                rows = list(csv.DictReader(f))
            dilute = [r for r in rows if r["model"] == "eshelby_dilute" and float(r["Vf"]) > .1]
            self.assertTrue(dilute)
            self.assertTrue(all(r["E"] == "" and r["note"] for r in dilute))

    def test_single_run_works_from_another_directory(self):
        with tempfile.TemporaryDirectory() as d:
            result = subprocess.run([sys.executable, str(ROOT/"main.py"), "--vf", ".05",
                                     "--output", d], cwd=d, capture_output=True, text=True)
            self.assertEqual(result.returncode, 0, result.stderr)
            payload = json.loads((Path(d)/"stiff_inclusions_single.json").read_text())
            self.assertEqual(len(payload["results"]), 10)

    def test_cli_bad_case_is_readable(self):
        result = subprocess.run([sys.executable, str(ROOT/"main.py"), "--case", "missing"],
                                capture_output=True, text=True)
        self.assertEqual(result.returncode, 2)
        self.assertIn("Unknown case", result.stderr)

    def test_cli_rejects_inconsistent_volume_fractions(self):
        result = subprocess.run([sys.executable, str(ROOT/"main.py"), "--vf", ".2", "--vm", ".7"],
                                capture_output=True, text=True)
        self.assertEqual(result.returncode, 2)
        self.assertIn("Vf + Vm", result.stderr)


if __name__ == "__main__":
    unittest.main()
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Imports CSV reading to verify what was actually written to disk.
L2	Imports JSON reading for saved single-run results.
L3	Imports filesystem paths.
L4	Imports launching of independent Python processes.
L5	Provides the current Python executable path so subprocesses use the same interpreter.
L6	Imports temporary directories that are cleaned up automatically.
L7	Imports the testing framework.
L9	Imports comparison-grid/evaluation/export functions under integration test.
L10	Imports material records and the JSON database loader.
L12	Finds the project folder from this test file's location rather than assuming a terminal working directory.
L15	Defines the project-workflow test collection.
L16	Marks the class-wide setup as a class method.
L17	Defines one setup operation shared by all tests in this collection.
L18	Loads the included illustrative stiff-inclusion case for the tests.
L20	Defines the metadata/provenance check for shipped demonstration data.
L21	Visits every shipped material case.
L22	Checks that none has been mislabeled as measured or literature data.
L23	Requires a nonempty explanation of the input source/assumptions.
L25	Defines schema and unsupported-geometry rejection checks.
L26	Converts the reference Material to a mutable dictionary for deliberately bad variants.
L27	Removes the record-level unit field because from_dict receives the shared unit separately.
L28	Visits three invalid changes: cylinder geometry, an unknown misspelled field, and a negative E.
L29	Expects each bad variant to raise ValueError.
L30	Combines the original fields with the bad change and attempts validation.
L32	Defines the key-fraction grid test.
L33	Requests a coarse grid that would not naturally contain every reporting fraction.
L34	Visits the required summary fractions within the extent.
L35	Checks that each was inserted.
L36	Checks the returned grid is sorted and has no duplicate values.
L38	Defines invalid sweep configuration checks.

Source line(s)	Explanation and connection to the project
L39	Visits bad maxima and unsuitable grid sizes.
L40	Expects each configuration to fail validation.
L41	Attempts to generate the invalid grid.
L43	Defines the policy check for dilute rows outside the display interval.
L44	Evaluates a case at Vf=.3, beyond that configured interval.
L45	Finds its dilute row using a generator and next.
L46	Checks that unavailable E is a blank, not zero or an extrapolated result.
L47	Checks that the status explains why no value was generated.
L48	Checks that even unavailable results retain their row, preserving ten rows per fraction.
L50	Defines comparison-layer handling of unsupported K/G ordering.
L51	Copies the reference material's data.
L52	Removes its unit for the same factory-interface reason as above.
L53	Comment describing the intentionally constructed cross-ordering.
L54	Creates a stable material pair where larger E does not imply larger K, demonstrating why ordering only by E is wrong.
L55	Evaluates it at a low fraction so the other models can still run.
L56	Selects the two HS bound rows.
L57	Checks both expected bound labels are retained.
L58	Checks both are marked unavailable with blank E values instead of fabricated bounds.
L60	Defines a round-trip CSV check of missing results and interpretation notes.
L61	Creates a temporary directory for the exported table.
L62	Computes a small sweep containing fractions above .1 and writes it to CSV.
L63	Opens the generated file for verification.
L64	Reads it back as dictionaries, testing serialization rather than just in-memory rows.
L65	Finds dilute entries above the configured display cutoff; CSV fraction strings are converted back to floats.
L66	Checks that such entries actually exist, preventing a vacuous all(...) test.
L67	Checks every such entry preserved both its missing-value marker and explanatory note.
L69	Defines a portability test for launching from outside the project folder.
L70	Creates a temporary working/output directory.
L71-72	Starts main.py with the current interpreter, an explicit .05 fraction and a temporary output folder, capturing text output and errors.
L73	Checks successful process exit, displaying captured stderr if it failed.

Source line(s)	Explanation and connection to the project
L74	Reads the actual JSON output from that separate run.
L75	Checks that all ten expected result rows were saved.
L77	Defines error-message behavior for an unknown material id.
L78-79	Starts the command-line program with a case called missing and captures its output.
L80	Checks it exits with the intended handled-error status 2.
L81	Checks the user-facing error says the case is unknown.
L83	Defines the contradictory-fraction CLI check.
L84-85	Runs main.py with Vf=.2 and Vm=.7, capturing output.
L86	Checks the program rejects the input with error status 2.
L87	Checks the explanation refers to the violated Vf+Vm constraint.
L90	Checks whether the integration-test file is run directly.
L91	Starts unittest for this file if so.

utilities/__init__.py

Source file: `utilities/__init__.py`

Purpose

This package entry point exposes public conversion helpers. It adds no new physics and does not run a material study during import.

Numbered source

```
001 | """Shared material, validation and tensor tools."""
002 | from .elastic_properties import to_bulk_shear, from_bulk_shear
003 |
004 | __all__ = ["to_bulk_shear", "from_bulk_shear"]
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Describes the utilities package's shared responsibilities.
L2	Imports the public forward and inverse conversion names into the package namespace.
L4	Lists the two public wildcard-import names, enabling a compact utilities interface.

analysis/__init__.py

Source file: `analysis/__init__.py`

Purpose

This file identifies the analysis package and documents its responsibility. The actual orchestration and plotting code lives in its separate modules.

Numbered source

```
001 | """Comparison, plotting and reproducible result export."""
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Package docstring describing comparison, plotting and reproducible export. It performs no numerical calculation.

tests/__init__.py

Source file: `tests/__init__.py`

Purpose

This supporting package marker gives the command for repeatable test discovery. The test functions are implemented in the three test modules, not here.

Numbered source

```
001 | """Run with: python -m unittest discover -s tests -v"""
```

Line commentary

Source line(s)	Explanation and connection to the project
L1	Documents the unittest discovery command; this string does not execute the command by itself.

6. A complete numerical trace: following one calculation

Use the default illustrative case:

```
Ef = 70.0       # GPa
Em = 3.5        # GPa
nu_f = 0.20
nu_m = 0.35
Vf = 0.05
Vm = 0.95
```

Step A: validate, then convert the phases

`load_materials` checks the JSON schema and builds a Material. `evaluate` receives that record and the requested Vf. Shared validators check positivity, finite numbers, Poisson-ratio stability and Vf+Vm=1.

```
Kf = 70/[3(1-2*0.20)]   = 38.888889 GPa
Gf = 70/[2(1+0.20)]     = 29.166667 GPa
Km = 3.5/[3(1-2*0.35)]  =  3.888889 GPa
Gm = 3.5/[2(1+0.35)]    =  1.296296 GPa
```

Step B: calculate the simple mixture reference values

```
E_equal_strain = 0.05*70 + 0.95*3.5 = 6.825000 GPa
E_equal_stress = 1/(0.05/70 + 0.95/3.5) = 3.674541 GPa
```

These calculations assume idealized equal strain or equal stress. They are not two measurements and they do not, on their own, identify the actual microstructure of a specimen.

Step C: build the Eshelby and concentration tensors

At matrix nu=.35, S has hydrostatic eigenvalue .692307692 and five equal deviatoric eigenvalues .461538462. Because both phases are isotropic, J and D also diagonalize the phase stiffness and concentration tensors. The six-by-six linear solve is therefore equivalent to two scalar sphere concentration factors:

```
zeta_m = Gm(9Km+8Gm)/[6(Km+2Gm)] = 1.512345679 GPa
aK = (Km+4Gm/3)/(Kf+4Gm/3) = 0.138297872
aG = (Gm+zeta_m)/(Gf+zeta_m) = 0.091549296
A_dil = aK J + aG D
```

The inclusion's strain concentration is smaller than one because it is stiffer than the matrix in this example. This is a predicted strain ratio, not a volume fraction or a modulus. The dilute effective calculation uses these factors directly and gives E=3.831403829 GPa.

Step D: normalize for Mori-Tanaka and recover E

```
B = (Vm+Vf*aK)J + (Vm+Vf*aG)D
Af = [aK/(Vm+Vf*aK)]J + [aG/(Vm+Vf*aG)]D
K_MT = Km + Vf*(Kf-Km)*aK/(Vm+Vf*aK)
G_MT = Gm + Vf*(Gf-Gm)*aG/(Vm+Vf*aG)

K_MT = 4.141807177 GPa
G_MT = 1.429942484 GPa
E_MT = 9*K_MT*G_MT/(3*K_MT+G_MT) = 3.847095921 GPa
nu_MT = 0.345192539
```

This traces `main -> evaluate -> mori_tanaka -> dilute_concentration -> sphere_eshelby -> properties_from_stiffness -> properties`. Notice that Mori-Tanaka does not simply average Ef and Em.

Step E: solve self-consistent equations instead

The same input phases are used, but each phase's surrounding host is now the unknown effective material. The bulk equation is eliminated at every trial G, then logarithmic bisection reduces the shear residual. The default run gives:

```
K_SC = 4.148971802 GPa
G_SC = 1.436473312 GPa
E_SC = 3.863536956 GPa
nu_SC = 0.344799421
iterations = 30
maximum dimensionless residual = 1.7590e-10
```

The residual being small establishes that the implemented equations are satisfied. It does not show that E_SC is experimentally accurate to 1e-10.

Step F: interpret the complete ten-row result

Output label	E (GPa)	Meaning
simple_voigt	6.825000	1D equal-strain E mixture
simple_reuss	3.674541	1D equal-stress E mixture
isotropic_vr_lower	3.674541	Proper isotropic compliance lower bound
isotropic_vr_upper	6.962397	Proper isotropic stiffness upper bound
eshelby_dilute	3.831404	Low-concentration sphere estimate
mori_tanaka	3.847096	Average-matrix-host prediction
self_consistent	3.863537	Effective-medium prediction
hs_lower	3.847096	Isotropic lower bound for this ordered phase pair
hs_upper	5.374874	Isotropic upper bound for this ordered phase pair
halpin_tsai_demo	3.973872	Separate uncalibrated xi=2 parameter demonstration

The simple Reuss E value agrees with the isotropic Reuss E here; the important distinction is that proper isotropic bounds supply a full K/G pair and the arithmetic simple E upper idealization is not generally the proper isotropic Voigt E bound. MT/HS lower equality is expected for this soft-matrix spherical case. Dilute can lie below the HS interval because it is a truncated low-concentration approximation; do not present it as an exact finite-fraction solution guaranteed to lie inside every rigorous bound.

7. Understanding outputs/figures and outputs/tables

Neither folder contains a new physical model. They store the products of `compare_models.py` and `plotting.py`. Deleting generated figures/tables does not delete equations; running the relevant command recreates them.

Figure files

For each case id there are two chart names, each saved as PNG and PDF:

- `<id>_isotropic_comparison`: isotropic VR and supported HS bands, MT and SC.
- `<id>_model_demonstrations`: dilute/MT/SC at low fractions in one panel, and simple E mixtures/HT in the other panel.

PNG is easy to view or put in a document. Vector PDF is useful for resizing without rasterizing graph lines and text. Neither file format changes the numbers. Transparency is visual layering, not a statistical confidence level. The axes are Vf (dimensionless) and E in the declared common modulus unit.

Table files

- `<id>_all_results.csv`: every model/bound row at every sweep fraction.
- `<id>_key_fractions.csv`: only standard reporting fractions from that sweep.
- `all_cases_results.csv`: all selected cases combined.
- `<id>_single.csv`: the ten rows from a single-run command, when generated.

Column	How to read it
case	Material identifier matching the data file
unit	Common unit for E,K,G; it does not apply to nu or Vf
Vf, Vm	The inclusion and matrix volume fractions
model	The exact calculation or bound label
E	Main Young's-modulus result, or blank if unavailable
nu	Effective/bounding-tensor Poisson ratio, when calculated
K, G	Effective/bounding bulk and shear moduli, when calculated
status	Whether the row is a calculation, idealization, demonstration or unavailable result
note	Assumptions and the reason for absence or separate interpretation
residual_norm	SC's maximum absolute dimensionless equation residual
iterations	SC's logarithmic bisection count; exact endpoints need zero

`ok` is the program's default successful-calculation label, not a statement of experimental validity. `low_fraction` labels the configured dilute display interval. `idealization` labels the 1D E mixtures. `parameter_demo` marks HT. `not_evaluated` and `not_applicable` are deliberately blank numerical results. E-only estimates leave nu,K,G blank; do not infer nonexistent values from zero.

With the default three cases and 61 fractions, there are `3*61*10=1830` result rows in the combined CSV (plus its column header). Changing the point count may produce more rows than points*10 because key fractions are inserted automatically. The code's progress count uses the actual row count divided by ten.

Other generated outputs

`run_manifest.json` records the actual selected inputs, source data location, sweep settings, plot paths and scope. It is a useful reproducibility record, but not a cryptographic record of source code, OS or every dependency version. `verification.txt` contains the real test runner output. `verification.json` stores the pass status and counts. `outputs/report` contains generated discussion and Markdown/PDF reports.

8. The report folder beyond the originally pasted tree

Your complete project also has `report/build_report.py`, a reporting utility called only with `--report`. It does not introduce another homogenization law. This guide's line-by-line coverage follows the files requested in your tree plus their core supporting modules; the PDF layout builder is summarized here rather than included as another long source chapter.

- `verify_project` starts unittest in a separate process, saves its actual output/status, and blocks report creation on a failing suite.
- `_value` retrieves one already-computed property at a specific fraction; it does not interpolate missing rows.
- `_discussion` builds prose and tables from actual result rows.
- `build_report` combines report sources and computed results, and writes Markdown and PDF deliverables.
- `_write_pdf` lays out headings, paragraphs, charts, tables, references and page footers with ReportLab. Its formatting choices do not change E.
- `theory.md`, `background_and_viva.md`, and `references.md` are readable source text. `discussion.md` is regenerated from the selected results.

9. What you should be able to explain in your presentation

1. Your objective is effective elastic stiffness under specified assumptions.
2. Ef,Em,Vf,Vm suffice for simple scalar mixtures; the full sphere models also need nu_f and nu_m.
3. K controls volume change, G controls shape change, and together they determine isotropic E and nu.
4. Kelvin 6x6 arrays represent fourth-order tensor operations with consistent shear scaling; they are not finite-element meshes.
5. S is the Eshelby constraint tensor; A is a different strain-concentration tensor that also incorporates constituent stiffness contrast.
6. Dilute neglects interactions, MT uses the average matrix host, and classical SC uses the unknown effective host. These assumptions explain differing E.
7. HS gives bounds, not a uniquely predicted material property, and phase order must be checked in K and G rather than only E in this implementation.
8. HT needs xi justification; an uncalibrated curve is not an experimentally supported prediction of your chosen geometry.
9. Code verification checks equations and execution. Experimental validation checks agreement with actual data and has not been performed here.
10. The real extensions are sourced material inputs, geometry/anisotropy that matches the specimen, and validation measurements. A woven classification paragraph does not mean the program has computed woven stiffness.

A practical study exercise

Run the default case, reproduce the two simple mixture values by hand, then verify the phase conversions. Read tensor_operations and Eshelby next, focusing on the meaning of the variables before the matrix syntax. Run at Vf=.01,.05 and .1 and compare dilute/MT/SC. Finally try the soft_inclusions case and explain why adding the second phase reduces E. Keep HT on its separate demonstration plot until you have a justified xi for the physical question you are asking.

10. Sources for further study

The implementation and this guide are tied to the delivered source files. For the underlying mechanics, use the project's references, including:

- Eshelby's original inclusion paper.
- Echoes developer documentation: homogenization scheme equations.
- NPTEL: self-consistent method and the effective surrounding medium.
- NPTEL: Halpin-Tsai and geometry/loading dependence.
- RockPhyPy developer documentation: independent HS formula reference.

The derivations and worked trace in this guide explain what this particular implementation does. They do not replace geometry-specific theory for an anisotropic or woven composite, nor supply a literature material dataset.

11. Source-version and coverage record

Line numbers refer to the source version identified below. If you later edit the code, those line numbers can shift even if the equations remain the same. Blank lines are retained in the source listings but need no calculation note. Wrapped expressions are covered as one logical statement with a line range.

File	Source lines	Nonblank lines covered	SHA-256 (first 12 characters)
requirements.txt	5	5	37ffb1add1cc
README.md	200	170	d5be4a8d19c7
utilities/input_checks.py	43	33	87bb6ab4464c
utilities/elastic_properties.py	39	27	a55a71661b65
utilities/tensor_operations.py	28	21	eaba08ed1015
models/_common.py	4	4	96e4ccbabf6f
models/rule_of_mixtures.py	43	31	ca9aea8e2996
models/eshelby.py	78	64	27c63333a337
models/mori_tanaka.py	36	28	298302647284
models/self_consistent.py	89	75	f9f86c96b793
models/hashin_shtrikman.py	46	38	8b88bb00a878
models/halpin_tsai.py	38	31	018576e83d94
models/__init__.py	21	18	152ccb6f45d3
data/materials.json	42	42	3f2b654c6d01
utilities/materials.py	75	67	6505a451114c
analysis/compare_models.py	93	76	30c6a607d09d
analysis/plotting.py	85	73	a14d981e262f
main.py	102	94	71d6e23e4953
tests/test_limits.py	93	78	d035de71eeb6
tests/test_benchmarks.py	72	65	2d25a0eecff5
tests/test_project.py	91	75	d6e4bbff5bb4
utilities/__init__.py	4	3	b8fab7ab81d8
analysis/__init__.py	1	1	0c3f1a7fcc39
tests/__init__.py	1	1	1d225b5e5cf1

Coverage check: **24 files**, **1120 nonblank source lines accounted for**. This is a completeness check of the explanation, not a new physics-validation test.